Tarefa 11: Particionamento de Dados e Balanceamento de Carga com OpenMP

Matheus Bezeril Pinto

Outubro de 2026

Resumo

Este relatório apresenta uma simulação bidimensional de difusão viscosa implementada em C e paralelizada com OpenMP. O modelo retém apenas o termo viscoso da evolução da velocidade, utiliza diferenças finitas no espaço, Euler explícito no tempo e condições de contorno periódicas. A validação considera estados parado, uniforme e com perturbação gaussiana. Para a análise de desempenho, foram comparadas versões com e sem `collapse(2)`, utilizando os schedules `static`, `dynamic,8` e `guided,8`, com uma, duas e quatro threads solicitadas. Os testes utilizaram uma malha de 400×400 células, 5.000 passos e três repetições por configuração, totalizando 54 medições. O menor tempo médio foi de aproximadamente 1,762 s, obtido sem `collapse(2)`, com quatro threads e `guided,8`, correspondendo a um speedup de 3,267 sobre a referência comum. O uso de `collapse(2)` com `dynamic,8` apresentou custo elevado de escalonamento. Os resultados mostram que a granularidade do trabalho deve ser considerada junto à escolha do schedule.

1 Introdução

A paralelização de laços exige identificar quais iterações podem ser executadas de forma independente e como distribuir esse trabalho entre as threads. O OpenMP permite expressar essa distribuição por diretivas e cláusulas, como `schedule` e `collapse` [1].

Nesta tarefa, cada célula de uma malha possui duas componentes de velocidade, u e v. O estado seguinte de uma célula depende de seu valor atual e de quatro vizinhos. Embora exista dependência entre passos de tempo consecutivos, as células de um mesmo passo podem ser calculadas em paralelo quando os campos de entrada e saída são separados.

O objetivo é verificar a correção dessa evolução e comparar o efeito das diferentes distribuições de trabalho. A implementação foi mantida próxima à versão sequencial em `Old/`, preservando os modos de inicialização, os diagnósticos e a visualização em Python. Os fontes e os resultados analisados correspondem à revisão `5192499` do repositório [2].

2 Modelo e discretização

2.1 Difusão viscosa

Ao considerar somente os efeitos viscosos com viscosidade cinemática constante ν, o modelo utilizado é

$$\frac{\partial u}{\partial t} = \nu \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right), \tag{1}$$

$$\frac{\partial v}{\partial t} = \nu \left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} \right). \tag{2}$$

Não há pressão, forças externas ou transporte advectivo. Assim, trata-se de um modelo de difusão das componentes da velocidade; o programa não resolve o sistema geral de Navier–Stokes nem inclui uma projeção para impor incompressibilidade.

O domínio é $[0, L_x) \times [0, L_y)$, com $L_x = L_y = 1$. Para N_x e N_y células, definem-se

$$\Delta x = \frac{L_x}{N_x}, \qquad \Delta y = \frac{L_y}{N_y}.$$

A velocidade é associada aos centros das células: $x_i = (i + 1/2)\Delta x$ e $y_j = (j + 1/2)\Delta y$.

2.2 Diferenças finitas e Euler explícito

Para uma componente genérica w, a atualização é

$$\begin{aligned} w_{i,j}^{n+1} = w_{i,j}^{n} &+ r_x \left(w_{i+1,j}^{n} - 2w_{i,j}^{n} + w_{i-1,j}^{n} \right) \\ &+ r_y \left(w_{i,j+1}^{n} - 2w_{i,j}^{n} + w_{i,j-1}^{n} \right), \end{aligned} \tag{3}$$

em que

$$r_x = \frac{\nu \Delta t}{\Delta x^2}, \qquad r_y = \frac{\nu \Delta t}{\Delta y^2}.$$

O esquema tem precisão de primeira ordem no tempo e segunda ordem no espaço. Para a difusão explícita em duas dimensões, a condição de estabilidade é

$$r_x + r_y \leq \frac{1}{2}.$$

O código define $\nu = 0{,}01$, $\alpha = 0{,}20$ e

$$\Delta t = \frac{\alpha \min(\Delta x^2, \Delta y^2)}{\nu}. \tag{4}$$

Consequentemente, $r_x + r_y \leq 2\alpha = 0{,}40$, atendendo à condição inclusive em malhas retangulares.

As condições de contorno são periódicas: o vizinho anterior ao índice zero é o último índice da dimensão, e o vizinho após o último índice é zero. O domínio não possui paredes nem uma fronteira de saída.

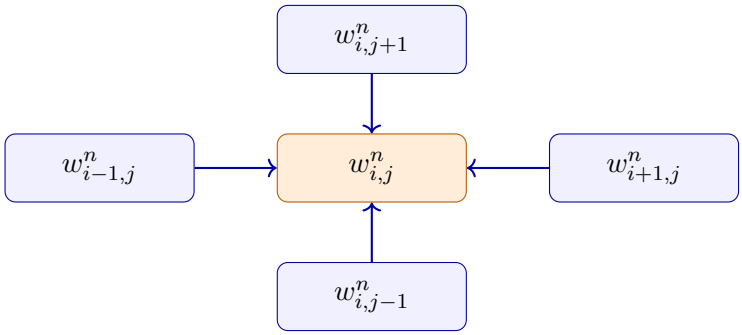


Figura 1: Estêncil de cinco pontos. A mesma atualização é aplicada separadamente a u e v. A separação entre campos atuais e novos permite calcular células diferentes em paralelo.

2.3 Condições iniciais

Os três modos de inicialização são:

1. Modo `0`: $u = v = 0$, representando o fluido parado.
2. Modo `1`: $u = 1$ e $v = 0$, representando velocidade uniforme.
3. Modo `2`: $v = 0$ e

$$u(x, y, 0) = 1 + 0{,}5 \exp\left[-\frac{(x - 0{,}5)^2 + (y - 0{,}5)^2}{2(0{,}08)^2}\right].$$

Nos dois primeiros modos, o Laplaciano é nulo e o estado deve permanecer constante. No terceiro, a gaussiana deve se espalhar e perder seu pico. A evolução por difusão não transporta o centro da perturbação como faria um termo advectivo.

3 Implementação

3.1 Organização dos arquivos e da memória

A implementação divide as responsabilidades em:

- `fluid.h`: parâmetros, estrutura `Simulation` e protótipos;
- `fluid.c`: alocação, inicialização, condições periódicas, evolução, diagnósticos e gravação;
- `fluid_sim.c`: seleção do modo e coordenação da execução;
- `visualize_fluid.py`: leitura dos CSVs e geração do GIF;
- `executar_testes.sh`: compilação e comparação das configurações OpenMP.

Os quatro vetores `u`, `v`, `u_new` e `v_new` são alocados com `calloc`. A posição (i, j) corresponde ao índice linear $iN_y + j$. Portanto, o laço interno em j percorre posições contíguas de memória.

Durante um passo, os vetores atuais são somente lidos e cada iteração escreve em uma posição distinta dos vetores novos. Não é necessário usar `atomic` ou `critical` nessa atualização. Para a carga de 400×400, os quatro campos ocupam aproximadamente 5,12 MB, sem contar estruturas e outros custos do processo.

3.2 Laços paralelos e troca dos campos

As duas versões são produzidas a partir do mesmo fonte:

Código 1: Seleção da diretiva OpenMP durante a compilação.

```
#ifdef USE_COLLAPSE
    #pragma omp parallel for collapse(2) schedule(runtime)
#else
    #pragma omp parallel for schedule(runtime)
#endif
    for (int i = 0; i < sim->nx; ++i)
    {
        for (int j = 0; j < sim->ny; ++j)
        {
            /* Leitura dos vizinhos e escrita da celula nova. */
        }
    }
```

O trecho acima resume a estrutura dos laços; o corpo completo está na cópia de `fluid.c` incluída na pasta `codigos/` do projeto do relatório.

Sem `collapse`, o espaço distribuído contém N_x iterações, cada uma responsável por uma linha com N_y células. Com `collapse(2)`, as duas dimensões formam um único espaço lógico com $N_x N_y$ iterações, cada uma correspondente a uma célula.

A opção `-DUSE_COLLAPSE` do GCC define a macro `USE_COLLAPSE` no pré-processador e seleciona a primeira diretiva. O prefixo `-D` é uma opção do compilador; não faz parte do nome da macro.

Ao final do `parallel for`, há uma barreira implícita. Somente depois que todas as células novas estão prontas, `swap_fields` troca os ponteiros dos campos atuais pelos novos. Essa troca evita copiar todos os valores e garante que o próximo passo leia um estado completo.

3.3 Schedules e granularidade

A cláusula `schedule(runtime)` seleciona a política em tempo de execução. Ela é definida pela variável `OMP_SCHEDULE`, sem recompilação:

- `static`: distribuição estática das iterações; não há obtenção dinâmica de novos blocos durante o laço;
- `dynamic,8`: as threads obtêm blocos de oito iterações conforme terminam o trabalho;

- `guided,8`: os blocos começam maiores e diminuem, com tamanho mínimo de oito, exceto um possível bloco final.

O trabalho por célula é essencialmente uniforme. Não há regiões em que uma célula execute um algoritmo mais caro que outra. Portanto, a distribuição dinâmica pode introduzir custo sem necessariamente corrigir um desequilíbrio relevante.

É importante interpretar a unidade do bloco: oito iterações significam oito linhas na versão normal, mas somente oito células na versão com `collapse(2)`.

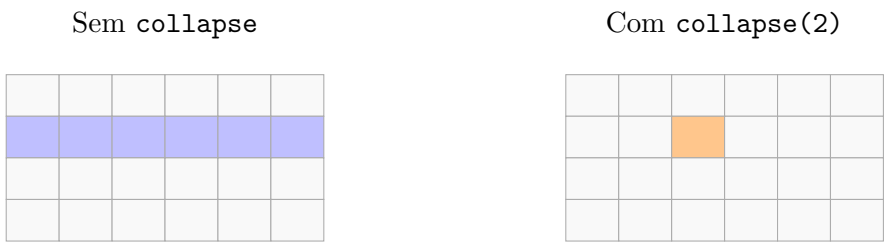


Figura 2: Unidade de distribuição ilustrada em uma malha reduzida. Com `dynamic,8`, a malha real de 400×400 distribui blocos de 3.200 células sem `collapse` e de oito células com `collapse(2)`.

3.4 Medição e modo de desempenho

A função `run_simulation` acumula o tempo apenas do cálculo e da troca dos campos:

Código 2: Trecho que delimita a medição em cada passo.

```
double inicio = omp_get_wtime();

compute_next_state(sim);
swap_fields(sim);

tempo_calculo += omp_get_wtime() - inicio;
```

Esse tempo inclui o custo do paralelismo dentro de `compute_next_state`, como a entrada e a saída da região paralela, o escalonamento e a sincronização. Não inclui a inicialização dos campos, a alocação, os diagnósticos, a gravação de CSVs ou a geração do GIF.

O argumento `-benchmark` define `save_every=0`, desativando frames, metadados de visualização e diagnósticos periódicos. Ele mantém a mesma evolução numérica. A saída ainda informa o tempo de cálculo para que o script o extraia.

4 Metodologia experimental

4.1 Ambiente

O hardware foi informado como o mesmo utilizado na Tarefa 10. A versão do compilador foi confirmada para os testes da Tarefa 11.

Tabela 1: Ambiente informado para as medições.

Componente	Especificação
Processador	Intel Core i5-6400, frequência nominal de 2,70 GHz
Núcleos / threads de hardware	4 núcleos físicos / 4 threads
Cache L1d / L1i	128 KiB / 128 KiB no total, quatro instâncias de cada
Cache L2 / L3	1 MiB / 6 MiB no total
Memória RAM	Aproximadamente 24 GiB
Sistema	Ubuntu Linux
Compilador	GCC 13.3.0, pacote Ubuntu `13.3.0-6ubuntu2~24.04.1`
Opções	`-O2 -fopenmp`, ligação com `-lm`

Não foram registrados controles de frequência da CPU, carga de outros processos ou afinidade das threads. A frequência nominal não significa frequência constante durante as execuções. Essas condições são consideradas na interpretação da variabilidade.

4.2 Duas cargas com finalidades diferentes

A visualização utiliza os parâmetros padrão. O desempenho utiliza uma carga maior para reduzir a influência relativa de medições muito curtas.

Tabela 2: Parâmetros das duas cargas analisadas.

Parâmetro	Validação visual	Desempenho
Modo	2	2
Malha	80×80	400×400
Passos	1.000	5.000
$\Delta x = \Delta y$	0,0125	0,0025
Δt	0,003125	0,000125
Tempo físico final	3,125	0,625
Atualizações de células	$6{,}4 \times 10^6$	$8{,}0 \times 10^8$
Frames	101, a cada 10 passos	Desativados

O segundo caso realiza 125 vezes mais atualizações de células, mas simula um tempo físico menor devido à redução de Δt. O tempo físico da tabela não é o tempo medido pelo relógio do computador.

4.3 Configurações e estatísticas

Foram comparadas duas versões, três quantidades de threads solicitadas $(1, 2, 4)$ e três schedules, totalizando 18 configurações. Cada configuração foi executada três vezes, gerando 54 medições:

Código 3: Comando que corresponde à configuração registrada nos resultados.

```
bash executar_testes.sh 2 3 400 400 5000
```

O script define `OMP_DYNAMIC=FALSE`, solicita o número de threads com `OMP_NUM_THREADS` e aplica cada schedule por `OMP_SCHEDULE`. Ele executa as configurações em ordem fixa, sem uma etapa de aquecimento separada. O número efetivo de threads não é gravado nos CSVs.

Para três tempos T_1, T_2, T_3, calculam-se

$$\overline{T} = \frac{1}{3}\sum_{k=1}^{3} T_k, \qquad s = \sqrt{\frac{1}{2}\sum_{k=1}^{3}(T_k - \overline{T})^2}, \qquad CV = 100\frac{s}{\overline{T}}.$$

As médias e os speedups são produzidos pelo script original. O desvio padrão amostral e o CV foram calculados adicionalmente para este relatório a partir dos mesmos tempos brutos. Nenhuma medição foi removida.

A referência comum é a média da versão normal com uma thread e `static`:

$$T_{\text{ref}} = 5{,}755341 \text{ s}, \qquad S = \frac{T_{\text{ref}}}{\overline{T}}.$$

Esse speedup mede a vantagem sobre uma configuração OpenMP de uma thread, e não sobre um executável sequencial compilado sem OpenMP. A pasta `Old/` foi usada como referência de correção.

5 Validação numérica

Nos modos parado e uniforme, as diferenças entre valores vizinhos são nulas. A Equação 3 preserva esses campos. Na verificação da implementação em ambiente de desenvolvimento, os 101 frames dos três modos foram comparados com os da versão sequencial de `Old/`; as versões normal e com `collapse(2)` produziram arquivos idênticos para os parâmetros padrão.

Os resultados de campo enviados ao repositório correspondem ao modo 2 na malha de 80×80. Os 101 frames dessa execução também coincidem, byte a byte, com os arquivos de referência em `Old/`. A análise desses frames não encontrou valores não finitos.

5.1 Decaimento e conservação da média

Para medir a perturbação no modo 2, utiliza-se $u_{\text{ref}} = 1$ e $v_{\text{ref}} = 0$:

$$A_{\max} = \max_{i,j} \sqrt{(u_{i,j} - 1)^2 + v_{i,j}^2}, \qquad A_{\text{RMS}} = \sqrt{\frac{1}{N_x N_y}\sum_{i,j}\left[(u_{i,j} - 1)^2 + v_{i,j}^2\right]}.$$

Tabela 3: Medidas extraídas dos frames inicial e final enviados ao repositório.

Medida	Inicial	Final
Máximo da perturbação	0,496958	0,046535
RMS da perturbação	0,070898	0,022755
Média de u	1,020106193	1,020106193
Variância de u	0,004622289	0,000113514
Máximo de $\lvert v \rvert$	0	0

O máximo e o RMS não crescem entre frames consecutivos, e a variância diminui. A maior variação da média de u ao longo dos arquivos é inferior a 4×10^{-14}, compatível com arredondamento numérico e a precisão dos CSVs.

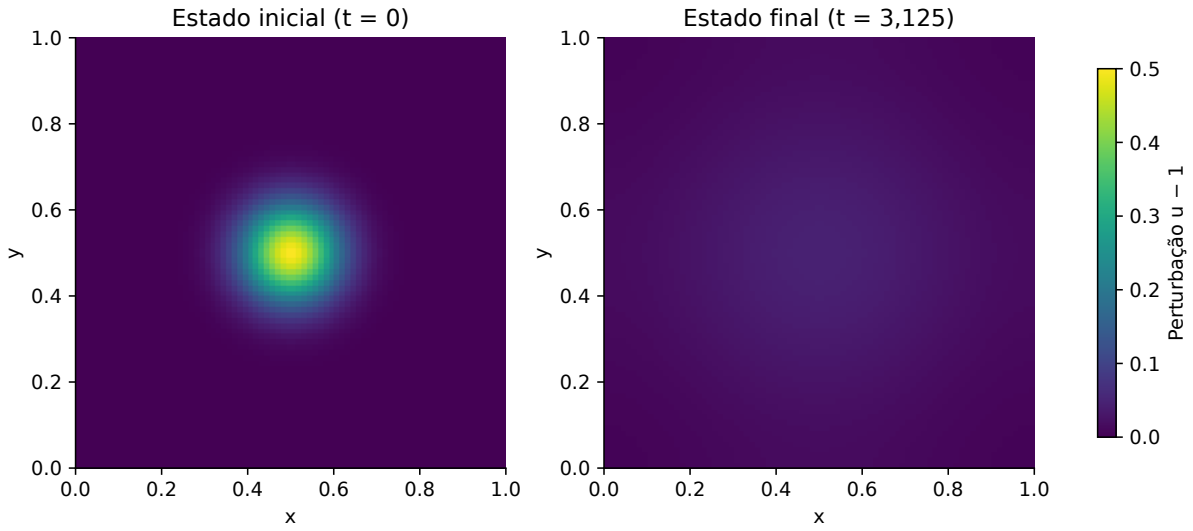


Figura 3: Perturbação inicial e final na mesma escala de cores, na malha de 80×80. A redução do pico e o espalhamento são compatíveis com a difusão.

Sob contornos periódicos, as contribuições dos vizinhos no Laplaciano se cancelam quando somadas sobre todo o domínio. Por isso, a média é conservada. A gaussiana inicial é positiva e acrescenta aproximadamente 0,020106193 à média de u; a tendência de longo prazo é um campo uniforme próximo de $u = 1{,}020106193$, e não exatamente $u = 1$.

Assim, o RMS medido em relação a $u = 1$ não precisa tender a zero. A variância em relação à própria média é uma medida complementar da uniformização. O estado final observado ainda não é perfeitamente uniforme.

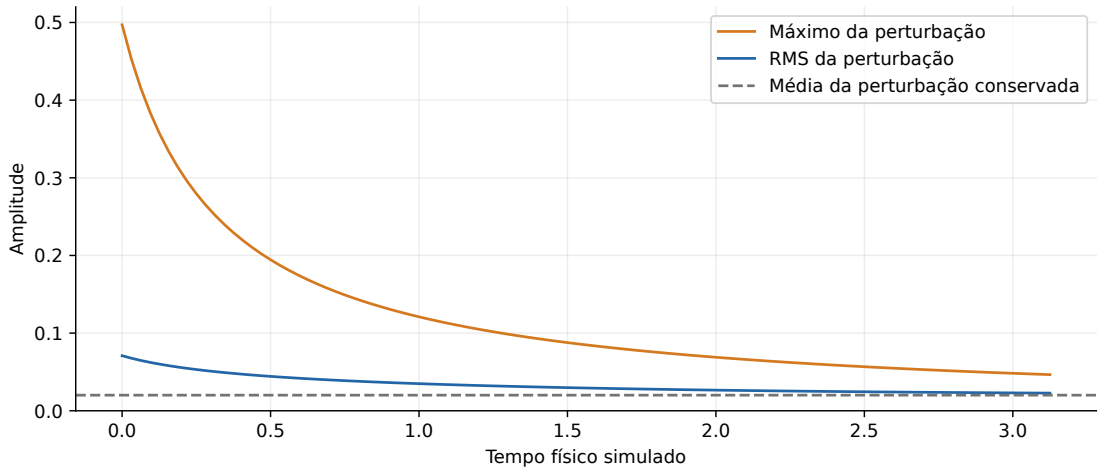


Figura 4: Decaimento do máximo e do RMS nos 101 frames. A linha tracejada representa a média positiva da perturbação que é conservada.

A concordância com `Old/` verifica a preservação do comportamento na paralelização. Ela não substitui um estudo de convergência de malha ou a comparação com uma solução analítica, que não fazem parte destes resultados.

6 Resultados de desempenho

A Tabela 4 reúne as 18 configurações. A diferença entre os tempos das repetições é mantida no desvio padrão, inclusive nos casos com maior dispersão.

Tabela 4: Resultados da carga de 400×400 células e 5.000 passos. s é o desvio padrão amostral; CV é o coeficiente de variação.

Versão	Schedule	Threads	Média (s)	s (s)	Speedup	CV (%)
Normal	`static`	1	5,755	0,026	1,000	0,45
Normal	`dynamic,8`	1	5,595	0,123	1,029	2,21
Normal	`guided,8`	1	5,451	0,095	1,056	1,74
Normal	`static`	2	2,899	0,029	1,985	1,01
Normal	`dynamic,8`	2	2,907	0,020	1,980	0,68
Normal	`guided,8`	2	2,801	0,020	2,055	0,73
Normal	`static`	4	2,054	0,420	2,801	20,43
Normal	`dynamic,8`	4	1,777	0,071	3,239	3,99
Normal	`guided,8`	4	1,762	0,038	3,267	2,15
Collapse	`static`	1	6,041	0,057	0,953	0,95
Collapse	`dynamic,8`	1	9,759	0,015	0,590	0,16
Collapse	`guided,8`	1	6,099	0,028	0,944	0,45
Collapse	`static`	2	3,211	0,040	1,792	1,23
Collapse	`dynamic,8`	2	9,505	0,118	0,606	1,25
Collapse	`guided,8`	2	3,155	0,040	1,824	1,28

Versão	Schedule	Threads	Média (s)	s (s)	Speedup	CV (%)
Collapse	`static`	4	2,225	0,347	2,587	15,59
Collapse	`dynamic,8`	4	6,890	0,226	0,835	3,28
Collapse	`guided,8`	4	1,943	0,024	2,962	1,25

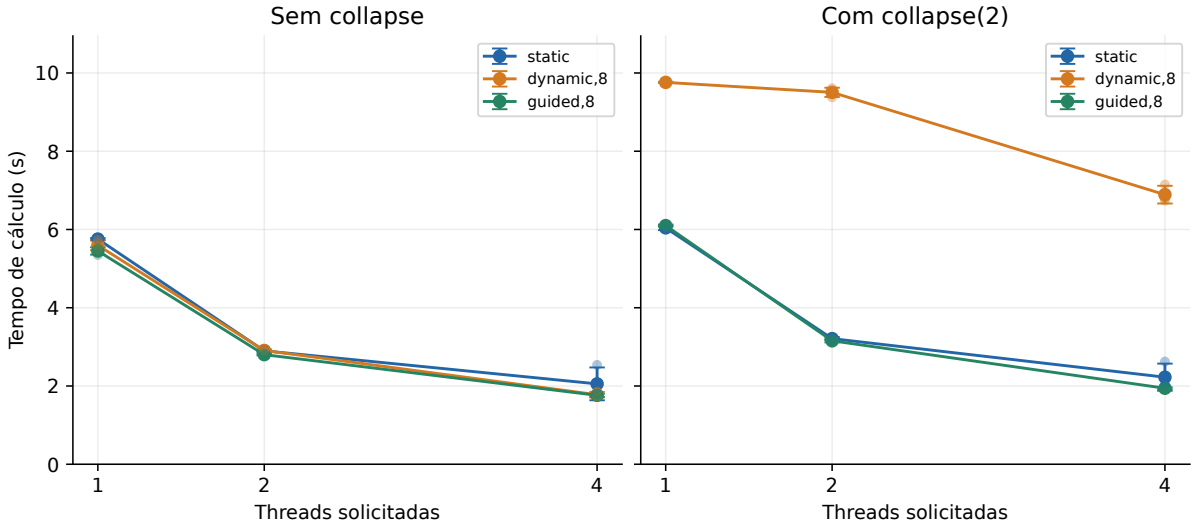


Figura 5: Tempos médios por configuração. As barras indicam um desvio padrão amostral e os pontos menores mostram as três medições. Não são intervalos de confiança.

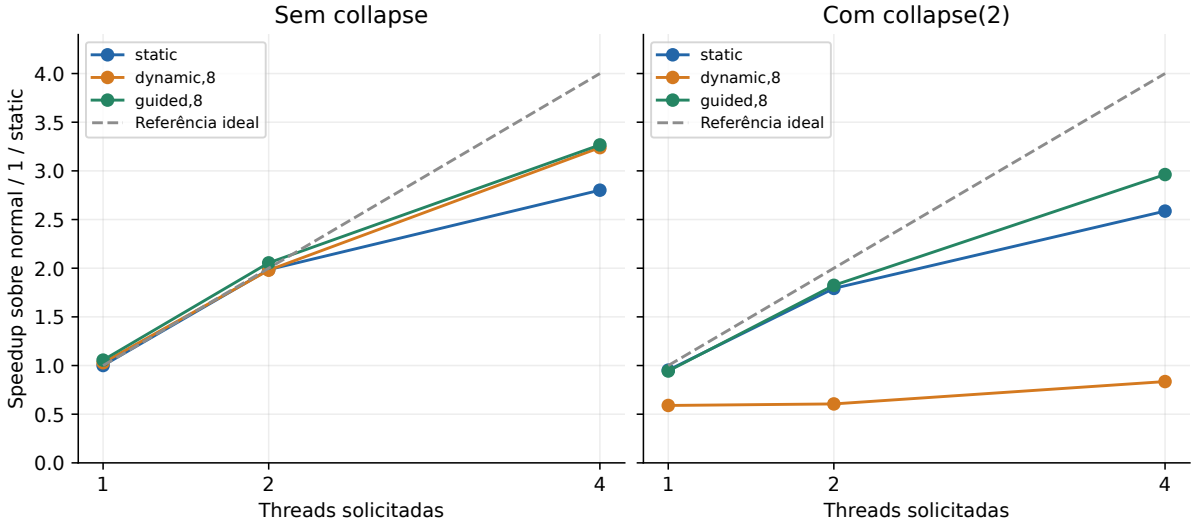


Figura 6: Speedup em relação à mesma referência normal / uma thread / `static`. A linha ideal é uma referência de escala, não um resultado medido.

O menor tempo médio foi obtido pela versão normal com quatro threads e `guided,8`: 1,761822 s, com speedup de aproximadamente 3,267. Com `dynamic,8`, a mesma versão obteve 1,777005 s e speedup de 3,239.

A melhor configuração com `collapse(2)` também utilizou quatro threads e `guided,8`, com média de 1,943000 s. Entre as médias deste experimento, nenhuma configuração com `collapse(2)` superou a configuração normal de mesmo schedule e mesma quantidade de threads.

7 Discussão

7.1 Escalabilidade e referência do speedup

Sem `collapse`, `static` passou de 5,755341 s com uma thread para 2,899031 s com duas, correspondendo a 1,985 vezes de aceleração. Esse resultado é próximo da redução ideal pela metade.

Para a melhor configuração de quatro threads, a razão $S/4$ é aproximadamente 81,7% em relação à referência comum. Esse percentual não deve ser confundido com utilização medida da CPU ou com eficiência relativa à versão sequencial de `Old/`.

O speedup 2,055 observado para duas threads com `guided,8` não comprova aceleração superlinear, pois a referência usa `static`. Ao comparar `guided,8` com seu próprio tempo de uma thread,

$$\frac{5{,}451358}{2{,}800924} \approx 1{,}946.$$

Portanto, diferenças entre schedules na configuração de uma thread também influenciam os speedups apresentados.

7.2 Por que `collapse(2)` não melhorou os tempos

A malha possui 400 linhas e foram solicitadas no máximo quatro threads. O laço externo já oferece iterações suficientes para a distribuição. Como o cálculo por célula é uniforme, aumentar o espaço lógico de 400 linhas para 160.000 células não cria, por si só, um benefício de balanceamento.

Além disso, o percurso em linhas respeita a disposição contígua dos dados. Alterar o particionamento pode modificar a localidade e os custos de execução. Os tempos confirmam a diferença de desempenho, mas, sem contadores de hardware, não permitem atribuir quantitativamente essa diferença a cache, divisão de índices ou largura de banda de memória.

O resultado não significa que `collapse` seja sempre inadequado. Ele pode ser útil quando o laço externo oferece poucas iterações em relação à quantidade de threads ou quando combinar as dimensões melhora a distribuição do trabalho.

7.3 Custo de `dynamic,8` com `collapse(2)`

Na versão normal, um bloco de oito iterações corresponde a oito linhas, ou 3.200 células. Por passo, as 400 linhas formam 50 blocos.

Com `collapse(2)`, as 160.000 células formam 20.000 blocos de oito células por passo. Ao longo de 5.000 passos, são aproximadamente 100 milhões de blocos, contra 250

mil na versão normal: 400 vezes mais unidades de distribuição.

Essa diferença de granularidade é compatível com o custo elevado observado. A versão com `collapse(2)` e `dynamic,8` levou 9,759254 s com uma thread e 6,890485 s com quatro. Há melhora ao aumentar suas próprias threads, mas a configuração de quatro threads continua mais lenta que a referência comum de uma thread.

O `guided,8` inicia com blocos maiores e reduz o número de obtenções de trabalho muito fino durante boa parte do laço. Isso é coerente com seus tempos menores. A explicação é baseada na política de distribuição e nas medições; não foi feito um perfil interno do runtime para separar cada custo.

7.4 Variabilidade e limites das conclusões

As configurações `static` com quatro threads apresentaram os maiores coeficientes de variação: 20,43% na versão normal e 15,59% com `collapse(2)`. Os tempos brutos foram, respectivamente,

$$(2{,}538934;\ 1{,}815439;\ 1{,}808791)\ \text{s}$$

e

$$(2{,}009164;\ 2{,}625166;\ 2{,}040834)\ \text{s}.$$

Os valores mais altos não foram descartados. Podem refletir condições do sistema, frequência ou escalonamento, mas o experimento não registrou evidências para determinar a causa.

A diferença entre `guided,8` e `dynamic,8` com quatro threads na versão normal é de aproximadamente 0,015 s, ou 0,85%. Como essa diferença é pequena diante da dispersão das três amostras, não há base suficiente para afirmar superioridade geral de um deles.

O uso de ordem fixa, a ausência de aquecimento separado e a pequena quantidade de repetições limitam a análise estatística. Ainda assim, as tendências mais marcantes, como o custo de `dynamic,8` com `collapse(2)`, aparecem de forma consistente nas amostras.

8 Conclusão

A implementação reproduziu o comportamento da versão sequencial nos modos analisados. Na execução com perturbação, o pico e o RMS diminuíram, a média foi conservada e a componente v permaneceu nula, em concordância com o modelo de difusão viscosa com contornos periódicos.

O aumento da carga permitiu obter tempos de segundos e observar aceleração mais clara. A versão normal atingiu speedup de aproximadamente 3,267 com quatro threads, enquanto `collapse(2)` não melhorou as médias nas condições avaliadas.

O principal aprendizado é que a escolha do schedule depende também da unidade de trabalho. O mesmo tamanho de bloco, oito, passou a significar oito células em vez de oito linhas com o uso de `collapse(2)`, aumentando muito o custo de distribuição dinâmica. Para este problema e esta quantidade de threads, paralelizar somente o laço externo foi suficiente.

As conclusões se restringem ao hardware, à carga e às amostras apresentados. Como continuidade, seria possível ampliar o número de repetições e variar o tamanho dos blocos dinâmicos, mantendo a mesma referência e a mesma carga.

A Reprodução das execuções

Os comandos a seguir são executados na pasta `Tarefa_11`. Para gerar os campos e o GIF:

```
gcc -O2 -fopenmp fluid_sim.c fluid.c -lm -o fluid_sim
OMP_NUM_THREADS=4 OMP_SCHEDULE=static ./fluid_sim 2

python3 -m venv .venv
source .venv/bin/activate
python -m pip install numpy matplotlib pillow
python visualize_fluid.py
```

O ambiente virtual é necessário apenas para as dependências Python. No Ubuntu, o pacote `python3-venv` deve estar instalado. O programa em C não depende de Python.

Para repetir a comparação de desempenho:

```
bash executar_testes.sh 2 3 400 400 5000
```

O script recompila as duas versões e grava `tempos.csv`, `resumo.csv` e `configuracao.txt` em `resultados_openmp/`. Uma nova execução substitui esses arquivos.

Para compilar manualmente a versão com `collapse(2)` e testar uma configuração:

```
gcc -O2 -fopenmp -DUSE_COLLAPSE \
    -DNX=400 -DNY=400 -DSTEPS=5000 \
    fluid_sim.c fluid.c -lm -o fluid_collapse
OMP_DYNAMIC=FALSE OMP_NUM_THREADS=4 OMP_SCHEDULE=guided,8 \
    ./fluid_collapse 2 --benchmark
```

B Material integral nos apêndices

O PDF é o documento de entrega. Por isso, os apêndices seguintes incorporam os fontes completos, os dados de desempenho, os diagnósticos e os metadados utilizados. Os arquivos são inseridos por `\lstinputlisting`, sem copiar manualmente cada linha para o texto LaTeX.

As cópias de `codigos/` registram a revisão analisada. A referência sequencial de `Old/` também está incluída. O script que gerou as figuras e tabelas aparece integralmente. As representações dos campos inicial e final estão no corpo do relatório; a evolução das medidas nos 101 frames está tabulada adiante.

C Fontes completos da implementação

C.1 `fluid.h`

```c
#ifndef FLUID_H
#define FLUID_H

#include <stddef.h>
#include <limits.h>

#ifndef NX
#define NX 80
#endif
#ifndef NY
#define NY 80
#endif

#define LX 1.0
#define LY 1.0

#define NU 0.01
#define ALPHA 0.20

#ifndef STEPS
#define STEPS 1000
#endif
#ifndef SAVE_EVERY
#define SAVE_EVERY 10
#endif

#if NX < 2 || NY < 2 || STEPS < 1 || SAVE_EVERY < 1
#error "Use NX e NY >= 2, STEPS e SAVE_EVERY >= 1."
#endif
#if NY > 0 && NX > INT_MAX / NY
#error "NX * NY deve caber em um indice do tipo int."
#endif

typedef struct
{
    int nx;
    int ny;

    double lx;
    double ly;

    double dx;
    double dy;
    double dt;

    double nu;
    double alpha;

    int steps;
    int save_every; // Zero desativa frames e diagnosticos nos testes de desempenho.
    int mode;

    double *u;
    double *v;

    double *u_new;
    double *v_new;

} Simulation;


// Inicializa os parâmetros e aloca os campos.
int init_simulation(Simulation *sim, int mode);

```

```c
// Libera a memória utilizada pela simulação.
void free_simulation(Simulation *sim);

// Define as condições iniciais do campo de velocidade.
void initialize_velocity(Simulation *sim);

// Cria o diretório dos frames.
int create_frames_directory(void);

// Salva os parâmetros usados pelo visualizador Python.
int save_metadata(const Simulation *sim);

// Salva o campo de velocidade atual em CSV.
int save_frame(const Simulation *sim, int step);

// Calcula e imprime as medidas da perturbação.
void print_diagnostics(const Simulation *sim, int step);

// Calcula o próximo estado usando diferenças finitas. (Ponto Chave)
void compute_next_state(Simulation *sim);

// Troca os campos atual e novo.
void swap_fields(Simulation *sim);

// Executa a evolução temporal da simulação.
int run_simulation(Simulation *sim);

// Imprime os parâmetros da simulação.
void print_simulation_info(const Simulation *sim);

// Retorna o nome do modo selecionado.
const char *get_mode_name(int mode);

#endif
```

C.2 fluid_sim.c

```c
#include "fluid.h"

#include <stdio.h>
#include <stdlib.h>
#include <string.h>


int main(int argc, char **argv)
{
    int mode = 2;

    if (argc > 3 ||
        (argc > 1 && (strlen(argv[1]) != 1 || argv[1][0] < '0' || argv[1][0] > '2')) ||
        (argc == 3 && strcmp(argv[2], "--benchmark") != 0))
    {
        fprintf(
            stderr,
            "Uso: %s [modo: 0, 1 ou 2] [--benchmark]\n",
            argv[0]
        );

        return EXIT_FAILURE;
    }

    if (argc > 1)
        mode = argv[1][0] - '0';

    // Inicializa a simulação.
    Simulation sim;
    if (!init_simulation(&sim, mode))
        return EXIT_FAILURE;

    if (argc == 3)
        sim.save_every = 0;

    // Define o estado inicial do fluido.
    initialize_velocity(&sim);

    // Cria o diretório dos frames.
    if (sim.save_every > 0 && !create_frames_directory())
    {
        free_simulation(&sim);
        return EXIT_FAILURE;
    }

    // Salva os parâmetros para o visualizador Python.
    if (sim.save_every > 0 && !save_metadata(&sim))
    {
        free_simulation(&sim);
        return EXIT_FAILURE;
    }

    // Exibe as configurações utilizadas.
    print_simulation_info(&sim);

    printf(
        "Modo selecionado: %s\n\n",
        get_mode_name(sim.mode)
    );

    // Executa a simulação.
    if (!run_simulation(&sim))
    {
        free_simulation(&sim);
        return EXIT_FAILURE;
    }

```

```
68      // Libera a memória utilizada.
69      free_simulation(&sim);
70
71      printf(
72          "\nSimulacao concluida.\n"
73      );
74
75      return EXIT_SUCCESS;
76  }
```

C.3 `fluid.c`

```c
#include "fluid.h"

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <sys/stat.h>
#include <errno.h>
#include <string.h>
#include <omp.h>


// Retorna o índice linear correspondente à posição (i,j).
static inline int index_of(const Simulation *sim, int i, int j)
{
    return i * sim->ny + j;
}


// Aplica condições de contorno periódicas.
static inline int periodic(int value, int size)
{
    if (value < 0)
        return value + size;

    if (value >= size)
        return value - size;

    return value;
}


// Libera toda a memória usada pela simulação.
void free_simulation(Simulation *sim)
{
    free(sim->u);
    free(sim->v);
    free(sim->u_new);
    free(sim->v_new);

    sim->u = NULL;
    sim->v = NULL;
    sim->u_new = NULL;
    sim->v_new = NULL;
}


// Aloca os campos de velocidade.
static int allocate_fields(Simulation *sim)
{
    size_t total =
        (size_t) sim->nx * sim->ny;

    sim->u = calloc(
        total,
        sizeof(double)
    );

    sim->v = calloc(
        total,
        sizeof(double)
    );

    sim->u_new = calloc(
        total,
        sizeof(double)
    );

```

```
68      sim->v_new = calloc(
69          total,
70          sizeof(double)
71      );
72
73      if (!sim->u || !sim->v ||
74          !sim->u_new || !sim->v_new)
75      {
76          fprintf(
77              stderr,
78              "Falha na alocacao de memoria.\n"
79          );
80
81          free_simulation(sim);
82
83          return 0;
84      }
85
86      return 1;
87  }
88
89
90  // Inicializa os parâmetros da simulação.
91  int init_simulation(Simulation *sim, int mode)
92  {
93      sim->nx = NX;
94      sim->ny = NY;
95
96      sim->lx = LX;
97      sim->ly = LY;
98
99      sim->nu = NU;
100     sim->alpha = ALPHA;
101
102     sim->steps = STEPS;
103     sim->save_every = SAVE_EVERY;
104     sim->mode = mode;
105
106     sim->dx = sim->lx / sim->nx;
107     sim->dy = sim->ly / sim->ny;
108
109     sim->dt =
110         sim->alpha *
111         fmin(sim->dx * sim->dx, sim->dy * sim->dy) /
112         sim->nu;
113
114     sim->u = NULL;
115     sim->v = NULL;
116     sim->u_new = NULL;
117     sim->v_new = NULL;
118
119     return allocate_fields(sim);
120 }
121
122
123 // Define as condições iniciais.
124 void initialize_velocity(Simulation *sim)
125 {
126     const double cx = 0.5;
127     const double cy = 0.5;
128     const double sigma = 0.08;
129
130     for (int i = 0; i < sim->nx; ++i)
131     {
132         double x =
133             (i + 0.5) * sim->dx;
134
135         for (int j = 0; j < sim->ny; ++j)
136         {
137             double y =
```

```
                (j + 0.5) * sim->dy;

            int current =
                index_of(sim, i, j);

            // Inicialmente o fluido está parado.
            sim->u[current] = 0.0;
            sim->v[current] = 0.0;

            // Os modos 1 e 2 possuem velocidade uniforme.
            if (sim->mode >= 1)
            {
                sim->u[current] = 1.0;
            }

            // O modo 2 adiciona uma perturbação gaussiana.
            if (sim->mode == 2)
            {
                double dx = x - cx;
                double dy = y - cy;

                double r2 =
                    dx * dx +
                    dy * dy;

                double perturbation =
                    0.5 *
                    exp(
                        -r2 /
                        (2.0 * sigma * sigma)
                    );

                sim->u[current] +=
                    perturbation;
            }
        }
    }
}


// Cria o diretório dos frames.
int create_frames_directory(void)
{
    if (mkdir("frames", 0755) != 0 &&
        errno != EEXIST)
    {
        fprintf(
            stderr,
            "Erro ao criar diretorio frames: %s\n",
            strerror(errno)
        );

        return 0;
    }

    return 1;
}


// Salva os parâmetros usados pelo Python.
int save_metadata(const Simulation *sim)
{
    FILE *file =
        fopen("metadata.txt", "w");

    if (!file)
    {
        fprintf(
            stderr,
            "Erro ao criar metadata.txt: %s\n",
```

```
            strerror(errno)
        );

        return 0;
    }

    fprintf(
        file,
        "NX %d\n",
        sim->nx
    );

    fprintf(
        file,
        "NY %d\n",
        sim->ny
    );

    fprintf(
        file,
        "LX %.12e\n",
        sim->lx
    );

    fprintf(
        file,
        "LY %.12e\n",
        sim->ly
    );

    fprintf(
        file,
        "NU %.12e\n",
        sim->nu
    );

    fprintf(
        file,
        "DT %.12e\n",
        sim->dt
    );

    fprintf(
        file,
        "STEPS %d\n",
        sim->steps
    );

    fprintf(
        file,
        "SAVE_EVERY %d\n",
        sim->save_every
    );

    fprintf(
        file,
        "MODE %d\n",
        sim->mode
    );

    fprintf(
        file,
        "TFINAL %.12e\n",
        sim->steps * sim->dt
    );

    fclose(file);

    return 1;
}
```

```
278
279
280 // Salva o campo atual em um arquivo CSV.
281 int save_frame(
282     const Simulation *sim,
283     int step
284 )
285 {
286     char filename[256];
287
288     snprintf(
289         filename,
290         sizeof(filename),
291         "frames/frame_%05d.csv",
292         step / sim->save_every
293     );
294
295     FILE *file =
296         fopen(filename, "w");
297
298     if (!file)
299     {
300         fprintf(
301             stderr,
302             "Erro ao abrir %s: %s\n",
303             filename,
304             strerror(errno)
305         );
306
307         return 0;
308     }
309
310     fprintf(
311         file,
312         "i,j,u,v\n"
313     );
314
315     for (int i = 0; i < sim->nx; ++i)
316     {
317         for (int j = 0; j < sim->ny; ++j)
318         {
319             int current =
320                 index_of(sim, i, j);
321
322             fprintf(
323                 file,
324                 "%d,%d,%.12e,%.12e\n",
325                 i,
326                 j,
327                 sim->u[current],
328                 sim->v[current]
329             );
330         }
331     }
332
333     fclose(file);
334
335     return 1;
336 }
337
338
339 // Calcula e imprime as medidas da perturbação.
340 void print_diagnostics(
341     const Simulation *sim,
342     int step
343 )
344 {
345     double max_perturbation = 0.0;
346     double sum_squared = 0.0;
347
```

```
348     double reference_u =
349         sim->mode >= 1 ? 1.0 : 0.0;
350
351     for (int i = 0; i < sim->nx; ++i)
352     {
353         for (int j = 0; j < sim->ny; ++j)
354         {
355             int current =
356                 index_of(sim, i, j);
357
358             double du =
359                 sim->u[current] -
360                 reference_u;
361
362             double dv =
363                 sim->v[current];
364
365             double amplitude =
366                 sqrt(
367                     du * du +
368                     dv * dv
369                 );
370
371             if (amplitude >
372                 max_perturbation)
373             {
374                 max_perturbation =
375                     amplitude;
376             }
377
378             sum_squared +=
379                 du * du +
380                 dv * dv;
381         }
382     }
383
384     double rms =
385         sqrt(
386             sum_squared /
387             ((double) sim->nx *
388              sim->ny)
389         );
390
391     double time =
392         step * sim->dt;
393
394     printf(
395         "step=%4d | "
396         "t=%8.4f | "
397         "max_pert=%10.6e | "
398         "RMS_pert=%10.6e\n",
399         step,
400         time,
401         max_perturbation,
402         rms
403     );
404 }
405
406
407 // Calcula o próximo estado por diferenças finitas. (Ponto Chave)
408 void compute_next_state(Simulation *sim)
409 {
410     double dx2 = sim->dx * sim->dx;
411     double dy2 = sim->dy * sim->dy;
412
413     // Permite comparar a paralelizacao com e sem collapse(2).
414 #ifdef USE_COLLAPSE
415     #pragma omp parallel for collapse(2) schedule(runtime)
416 #else
417     #pragma omp parallel for schedule(runtime)
```

```
418 #endif
419     for (int i = 0; i < sim->nx; ++i) // Loop sobre a dimensão x
420     {
421         for (int j = 0; j < sim->ny; ++j) // Loop sobre a dimensão y
422         {
423             int previous_i = periodic(i - 1, sim->nx);
424             int next_i = periodic(i + 1, sim->nx);
425             int previous_j = periodic(j - 1, sim->ny);
426             int next_j = periodic(j + 1, sim->ny);
427 
428             int current = index_of(sim, i, j);
429             int left = index_of(sim, previous_i, j);
430             int right = index_of(sim, next_i, j);
431             int down = index_of(sim, i, previous_j);
432             int up = index_of(sim, i, next_j);
433 
434             // Calcula o Laplaciano da componente u.
435             double laplacian_u = (
436                 (sim->u[right] - 2.0 * sim->u[current] + sim->u[left]) / dx2
437                 + (sim->u[up] - 2.0 * sim->u[current] + sim->u[down]) / dy2
438             );
439 
440             // Calcula o Laplaciano da componente v.
441             double laplacian_v = (
442                 (sim->v[right] - 2.0 * sim->v[current] + sim->v[left]) / dx2
443                 + (sim->v[up] - 2.0 * sim->v[current] + sim->v[down]) / dy2
444             );
445 
446             // Aplica Euler explícito à componente u.
447             sim->u_new[current] = sim->u[current] + sim->nu * sim->dt * laplacian_u;
448 
449             // Aplica Euler explícito à componente v.
450             sim->v_new[current] = sim->v[current] + sim->nu * sim->dt * laplacian_v;
451         }
452     }
453 }
454 
455 
456 // Troca os campos atual e novo.
457 void swap_fields(Simulation *sim)
458 {
459     double *temp;
460 
461     temp = sim->u;
462     sim->u = sim->u_new;
463     sim->u_new = temp;
464 
465     temp = sim->v;
466     sim->v = sim->v_new;
467     sim->v_new = temp;
468 }
469 
470 
471 // Executa a evolução temporal.
472 int run_simulation(Simulation *sim)
473 {
474     double tempo_calculo = 0.0;
475 
476     for (int step = 0;
477          step <= sim->steps;
478          ++step)
479     {
480         // Salva um frame nos intervalos definidos.
481         if (sim->save_every > 0 && step % sim->save_every == 0)
482         {
483             if (!save_frame(sim, step))
484                 return 0;
485 
486             print_diagnostics(
487                 sim,
```

```
488                 step
489             );
490         }
491
492         // Evita calcular um passo após o estado final.
493         if (step == sim->steps)
494             break;
495
496         // Mede somente o calculo, sem incluir a gravacao dos CSVs.
497         double inicio = omp_get_wtime();
498
499         // Calcula o próximo estado.
500         compute_next_state(sim);
501
502         // Torna o novo estado o estado atual.
503         swap_fields(sim);
504
505         tempo_calculo += omp_get_wtime() - inicio;
506     }
507
508     printf("Tempo de calculo: %.6f segundos\n", tempo_calculo);
509
510     return 1;
511 }
512
513
514 // Imprime as configurações da simulação.
515 void print_simulation_info(
516     const Simulation *sim
517 )
518 {
519     printf(
520         "Modo: %d\n",
521         sim->mode
522     );
523
524     printf(
525         "NX=%d NY=%d | "
526         "nu=%.5f | "
527         "dt=%.8f | "
528         "alpha=%.3f\n",
529         sim->nx,
530         sim->ny,
531         sim->nu,
532         sim->dt,
533         sim->alpha
534     );
535
536     printf(
537         "Tempo final: %.5f\n",
538         sim->steps * sim->dt
539     );
540
541     if (sim->save_every > 0)
542         printf("Arquivos salvos em ./frames/\n\n");
543     else
544         printf("Teste de desempenho: sem gravacao de frames.\n\n");
545 }
546
547
548 // Retorna uma descrição do modo utilizado.
549 const char *get_mode_name(int mode)
550 {
551     switch (mode)
552     {
553         case 0:
554             return "fluido parado";
555
556         case 1:
557             return "velocidade uniforme";
```

```
558
559         case 2:
560             return "velocidade uniforme + perturbacao";
561
562         default:
563             return "modo desconhecido";
564     }
565 }
```

C.4 executar_testes.sh

```bash
#!/usr/bin/env bash
# Tarefa 11: compara schedule e collapse(2) usando OpenMP.
# Uso: bash executar_testes.sh [modo] [repeticoes] [nx] [ny] [passos]
# Exemplo com mais carga: bash executar_testes.sh 2 3 400 400 5000

set -euo pipefail

PASTA=$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)
RESULTADOS="$PASTA/resultados_openmp"
MODO="${1:-2}"
REPETICOES="${2:-1}"
NX="${3:-80}"
NY="${4:-80}"
PASSOS="${5:-1000}"

if (( $# > 5 )); then
    echo "Uso: bash executar_testes.sh [modo] [repeticoes] [nx] [ny] [passos]" >&2
    exit 1
fi

if [[ ! "$MODO" =~ ^[012]$ ]]; then
    echo "Modo invalido. Use 0 (parado), 1 (uniforme) ou 2 (perturbacao)." >&2
    exit 1
fi
if [[ ! "$REPETICOES" =~ ^[1-9][0-9]*$ ]]; then
    echo "Quantidade de repeticoes invalida. Use um inteiro positivo." >&2
    exit 1
fi
if [[ ! "$NX" =~ ^[1-9][0-9]*$ || ! "$NY" =~ ^[1-9][0-9]*$ ||
      ! "$PASSOS" =~ ^[1-9][0-9]*$ ]] || (( NX < 2 || NY < 2 )); then
    echo "Use nx e ny inteiros >= 2 e passos inteiro positivo." >&2
    exit 1
fi
if ! command -v gcc >/dev/null 2>&1; then
    echo "GCC nao encontrado. Instale o compilador antes de executar." >&2
    exit 1
fi

mkdir -p "$RESULTADOS"

# Compila as duas versoes a partir dos mesmos arquivos-fonte.
echo "Compilando as versoes..."
DEFINICOES=("-DNX=$NX" "-DNY=$NY" "-DSTEPS=$PASSOS")
gcc -O2 -fopenmp "${DEFINICOES[@]}" "$PASTA/fluid_sim.c" "$PASTA/fluid.c" -lm -o "$PASTA/
    fluid_normal"
gcc -O2 -fopenmp -DUSE_COLLAPSE "${DEFINICOES[@]}" "$PASTA/fluid_sim.c" "$PASTA/fluid.c" -lm -o "
    $PASTA/fluid_collapse"

# Isola os logs das execucoes. O modo benchmark nao grava frames nem metadata.
TEMP_DIR=$(mktemp -d)
trap 'rm -rf -- "$TEMP_DIR"' EXIT

ARQUIVO_TEMPOS="$RESULTADOS/tempos.csv"
ARQUIVO_RESUMO="$RESULTADOS/resumo.csv"
printf 'versao;threads;schedule;repeticao;tempo_segundos\n' > "$ARQUIVO_TEMPOS"

THREADS=(1 2 4)
SCHEDULES=("static" "dynamic,8" "guided,8")
VERSOES=("normal" "collapse")

# O OpenMP deve respeitar a quantidade solicitada de threads.
export OMP_DYNAMIC=FALSE
export LC_ALL=C

# Registra a carga utilizada sem alterar o formato dos CSVs existentes.
printf 'MODO %s\nNX %s\nNY %s\nSTEPS %s\nREPETICOES %s\n' \
    "$MODO" "$NX" "$NY" "$PASSOS" "$REPETICOES" > "$RESULTADOS/configuracao.txt"
```

```
66
67  echo "Modo: $MODO | Malha: $NX x $NY | Passos: $PASSOS | Repeticoes: $REPETICOES"
68  for versao in "${VERSOES[@]}"; do
69      for threads in "${THREADS[@]}"; do
70          for schedule in "${SCHEDULES[@]}"; do
71              for ((r = 1; r <= REPETICOES; r++)); do
72                  # Mede o calculo sem o custo e o volume dos arquivos de visualizacao.
73                  (
74                      cd "$TEMP_DIR"
75                      OMP_NUM_THREADS="$threads" OMP_SCHEDULE="$schedule" \
76                          "$PASTA/fluid_$versao" "$MODO" --benchmark > "$TEMP_DIR/saida.log"
77                  )
78
79                  # O programa ja mede apenas compute_next_state + swap_fields.
80                  tempo=$(awk '/^Tempo de calculo:/ {print $(NF-1); exit}' "$TEMP_DIR/saida.log")
81                  if ! awk -v tempo="$tempo" 'BEGIN { exit !(tempo ~ /^[0-9]+[.][0-9]+$/ && tempo > 0)
     }'; then
82                      echo "Nao foi possivel ler o tempo da execucao." >&2
83                      exit 1
84                  fi
85
86                  printf '%s;%s;%s;%s;%s\n' \
87                      "$versao" "$threads" "$schedule" "$r" "$tempo" >> "$ARQUIVO_TEMPOS"
88                  printf '%-8s | %s threads | %-10s | repeticao %d | %s s\n' \
89                      "$versao" "$threads" "$schedule" "$r" "$tempo"
90              done
91          done
92      done
93  done
94
95  # Media dos tempos e speedup relativo a 1 thread, static, sem collapse.
96  awk -F';' '
97      NR == 1 { next }
98      {
99          chave = $1 SUBSEP $2 SUBSEP $3
100         if (!(chave in contagem)) {
101             ordem[++total] = chave
102             versao[chave] = $1
103             threads[chave] = $2
104             schedule[chave] = $3
105         }
106         soma[chave] += $5
107         contagem[chave]++
108     }
109     END {
110         print "versao;threads;schedule;tempo_medio_segundos;speedup"
111         base = "normal" SUBSEP "1" SUBSEP "static"
112         tempo_base = soma[base] / contagem[base]
113         for (i = 1; i <= total; i++) {
114             chave = ordem[i]
115             media = soma[chave] / contagem[chave]
116             printf "%s;%s;%s;%.6f;%.3f\n", \
117                 versao[chave], threads[chave], schedule[chave], \
118                 media, tempo_base / media
119         }
120     }
121 ' "$ARQUIVO_TEMPOS" > "$ARQUIVO_RESUMO"
122
123 echo
124 echo "Testes concluidos. Resultados:"
125 echo "  $ARQUIVO_TEMPOS"
126 echo "  $ARQUIVO_RESUMO"
127 echo "  $RESULTADOS/configuracao.txt"
128 echo "Nenhum frame foi gravado durante os testes."
```

C.5 visualize_fluid.py

```python
import glob
import os
import sys

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation


FRAME_DIR = "frames"
METADATA_FILE = "metadata.txt"


def read_metadata(filename):
    """Lê os parâmetros da simulação armazenados em metadata.txt."""
    meta = {}

    with open(filename, "r", encoding="utf-8") as f:
        for line in f:
            key, value = line.split()
            meta[key] = value

    return meta


def load_frame(filename, nx, ny):
    """Carrega um frame CSV e reconstrói os campos de velocidade u e v."""
    data = np.loadtxt(
        filename,
        delimiter=",",
        skiprows=1
    )

    u = data[:, 2].reshape(nx, ny)
    v = data[:, 3].reshape(nx, ny)

    return u, v


def progress_callback(current_frame, total_frames):
    """
    Exibe o progresso da geração do GIF no terminal.

    current_frame começa em 0, por isso é necessário somar 1
    para mostrar a numeração do frame de forma intuitiva.
    """
    frame = current_frame + 1
    percent = 100.0 * frame / total_frames

    print(
        f"\rGerando GIF: frame {frame}/{total_frames} "
        f"({percent:5.1f}%)",
        end="",
        flush=True
    )

    if frame == total_frames:
        print()


def main():
    # Permite informar uma pasta diferente para os frames pela linha de comando.
    frame_dir = os.path.normpath(
        sys.argv[1]
        if len(sys.argv) > 1
        else FRAME_DIR
    )
```

```
68
69      # O arquivo metadata.txt fica no mesmo diretório da pasta de frames.
70      metadata_path = os.path.join(
71          os.path.dirname(frame_dir) or ".",
72          METADATA_FILE
73      )
74
75      if not os.path.exists(metadata_path):
76          raise FileNotFoundError(
77              f"Arquivo '{metadata_path}' não encontrado. "
78              "Execute primeiro o programa em C."
79          )
80
81      # Lê os parâmetros da simulação.
82      meta = read_metadata(metadata_path)
83
84      nx = int(meta["NX"])
85      ny = int(meta["NY"])
86      lx = float(meta["LX"])
87      ly = float(meta["LY"])
88      dt = float(meta["DT"])
89      save_every = int(meta["SAVE_EVERY"])
90      nu = float(meta["NU"])
91      mode = int(meta["MODE"])
92      steps = int(meta["STEPS"])
93
94      # Localiza todos os arquivos CSV gerados pelo programa em C.
95      files = sorted(
96          glob.glob(
97              os.path.join(
98                  frame_dir,
99                  "frame_*.csv"
100             )
101         )
102     )
103     # Ignora sobras de uma execucao anterior com mais passos.
104     files = [file for file in files
105              if int(os.path.splitext(os.path.basename(file))[0].split("_")[-1])
106              <= steps // save_every]
107
108     if not files:
109         raise FileNotFoundError(
110             f"Nenhum frame encontrado em '{frame_dir}'. "
111             "Execute primeiro o programa em C."
112         )
113
114     print(f"{len(files)} frames encontrados.")
115
116     # Carrega os frames para a animação e para a análise da perturbação.
117     frames = [
118         load_frame(file, nx, ny)
119         for file in files
120     ]
121
122     # Calcula a magnitude da velocidade em cada frame.
123     speeds = [
124         np.hypot(u, v)
125         for u, v in frames
126     ]
127
128     # Calcula as coordenadas dos centros das células da malha.
129     x = (
130         np.arange(nx) + 0.5
131     ) * lx / nx
132
133     y = (
134         np.arange(ny) + 0.5
135     ) * ly / ny
136
137     X, Y = np.meshgrid(
```

```
        x,
        y,
        indexing="ij"
    )

    # Determina a maior magnitude de velocidade para manter
    # a mesma escala de cores durante toda a animação.
    vmax = max(
        float(np.max(speed))
        for speed in speeds
    )

    vmax = max(vmax, 1e-12)

    # Cria uma figura com o campo de velocidade e o gráfico da perturbação.
    fig, (ax_field, ax_decay) = plt.subplots(
        1,
        2,
        figsize=(13, 5)
    )

    # Mostra a magnitude da velocidade como imagem de fundo.
    image = ax_field.imshow(
        speeds[0].T,
        origin="lower",
        extent=[0, lx, 0, ly],
        aspect="equal",
        vmin=0,
        vmax=vmax,
        interpolation="nearest"
    )

    fig.colorbar(
        image,
        ax=ax_field,
        label="|v| (magnitude da velocidade)"
    )

    # Reduz a quantidade de setas para manter o campo vetorial legível.
    skip = max(1, nx // 20)

    quiver = ax_field.quiver(
        X[::skip, ::skip],
        Y[::skip, ::skip],
        frames[0][0][::skip, ::skip],
        frames[0][1][::skip, ::skip],
        angles="xy",
        scale_units="xy",
        scale=max(vmax / 0.15, 1.0)
    )

    ax_field.set_xlabel("x")
    ax_field.set_ylabel("y")
    ax_field.set_title("Campo de velocidade")

    # Define o campo uniforme de referência utilizado para medir
    # a intensidade da perturbação.
    background_u = 1.0 if mode >= 1 else 0.0

    # Calcula o instante correspondente a cada frame.
    times = (
        np.array([int(os.path.splitext(os.path.basename(file))[0].split("_")[-1])
                  for file in files])
        * save_every
        * dt
    )

    rms = []
    max_pert = []

```

```
    # Calcula o RMS e o valor máximo da perturbação em cada frame.
    for u, v in frames:
        du = u - background_u
        dv = v

        rms.append(
            float(
                np.sqrt(
                    np.mean(
                        du**2 + dv**2
                    )
                )
            )
        )

        max_pert.append(
            float(
                np.max(
                    np.hypot(
                        du,
                        dv
                    )
                )
            )
        )

    # Inicializa o gráfico do decaimento da perturbação.
    ax_decay.plot(
        times,
        rms,
        label="RMS da perturbação"
    )

    ax_decay.plot(
        times,
        max_pert,
        label="Máximo da perturbação"
    )

    # Marcadores que indicam a posição atual durante a animação.
    marker_rms, = ax_decay.plot(
        [times[0]],
        [rms[0]],
        marker="o",
        linestyle="None"
    )

    marker_max, = ax_decay.plot(
        [times[0]],
        [max_pert[0]],
        marker="o",
        linestyle="None"
    )

    ax_decay.set_xlabel("Tempo")
    ax_decay.set_ylabel("Amplitude da perturbação")
    ax_decay.set_title("Difusão da perturbação")
    ax_decay.grid(True, alpha=0.3)
    ax_decay.legend()

    # Define o texto correspondente ao modo da simulação.
    if mode == 0:
        mode_text = "fluido parado"
    elif mode == 1:
        mode_text = "velocidade uniforme"
    else:
        mode_text = "velocidade uniforme + perturbação"

    fig.suptitle(
        f"Difusão viscosa | nu={nu:g} | {mode_text}"
```

```
    )

    plt.tight_layout()

    def update(frame_number):
        """Atualiza o campo de velocidade e o gráfico da perturbação."""
        u, v = frames[frame_number]
        speed = speeds[frame_number]

        # Atualiza a imagem da magnitude da velocidade.
        image.set_data(speed.T)

        # Atualiza as setas do campo vetorial.
        quiver.set_UVC(
            u[::skip, ::skip],
            v[::skip, ::skip]
        )

        ax_field.set_title(
            f"Campo de velocidade | "
            f"frame {frame_number + 1}/{len(frames)}"
        )

        # Atualiza os marcadores do gráfico da perturbação.
        marker_rms.set_data(
            [times[frame_number]],
            [rms[frame_number]]
        )

        marker_max.set_data(
            [times[frame_number]],
            [max_pert[frame_number]]
        )

        return (
            image,
            quiver,
            marker_rms,
            marker_max
        )

    # Cria a animação usando todos os frames disponíveis.
    animation = FuncAnimation(
        fig,
        update,
        frames=len(frames),
        interval=60,
        blit=False,
        repeat=True
    )

    print(
        f"\nIniciando geração do GIF com "
        f"{len(frames)} frames..."
    )

    # Salva a animação e exibe o progresso no terminal.
    animation.save(
        "fluid_simulation.gif",
        writer="pillow",
        fps=15,
        progress_callback=progress_callback
    )

    print(
        "GIF salvo em: fluid_simulation.gif"
    )

    plt.close(fig)
```

```
348
349 if __name__ == "__main__":
350     main()
```

D Fontes completos da referência sequencial

D.1 `Old/fluid.h`

```c
#ifndef FLUID_H
#define FLUID_H

#include <stddef.h>

#define NX 80
#define NY 80

#define LX 1.0
#define LY 1.0

#define NU 0.01
#define ALPHA 0.20

#define STEPS 1000
#define SAVE_EVERY 10

typedef struct
{
    int nx;
    int ny;

    double lx;
    double ly;

    double dx;
    double dy;
    double dt;

    double nu;
    double alpha;

    int steps;
    int save_every;
    int mode;

    double *u;
    double *v;

    double *u_new;
    double *v_new;

} Simulation;


// Inicializa os parâmetros e aloca os campos.
int init_simulation(Simulation *sim, int mode);

// Libera a memória utilizada pela simulação.
void free_simulation(Simulation *sim);

// Define as condições iniciais do campo de velocidade.
void initialize_velocity(Simulation *sim);

// Cria o diretório dos frames.
int create_frames_directory(void);

// Salva os parâmetros usados pelo visualizador Python.
int save_metadata(const Simulation *sim);

// Salva o campo de velocidade atual em CSV.
int save_frame(const Simulation *sim, int step);

// Calcula e imprime as medidas da perturbação.
```

```c
void print_diagnostics(const Simulation *sim, int step);

// Calcula o próximo estado usando diferenças finitas. (Ponto Chave)
void compute_next_state(Simulation *sim);

// Troca os campos atual e novo.
void swap_fields(Simulation *sim);

// Executa a evolução temporal da simulação.
int run_simulation(Simulation *sim);

// Imprime os parâmetros da simulação.
void print_simulation_info(const Simulation *sim);

// Retorna o nome do modo selecionado.
const char *get_mode_name(int mode);

#endif
```

D.2 Old/fluid_sim.c

```c
#include "fluid.h"

#include <stdio.h>
#include <stdlib.h>


int main(int argc, char **argv)
{
    int mode = 2;

    if (argc > 1)
        mode = atoi(argv[1]);

    if (mode < 0 || mode > 2)
    {
        fprintf(
            stderr,
            "Modo invalido. Use 0, 1 ou 2.\n"
        );

        return EXIT_FAILURE;
    }

    // Inicializa a simulação.
    Simulation sim;
    if (!init_simulation(&sim, mode))
        return EXIT_FAILURE;

    // Define o estado inicial do fluido.
    initialize_velocity(&sim);

    // Cria o diretório dos frames.
    if (!create_frames_directory())
    {
        free_simulation(&sim);
        return EXIT_FAILURE;
    }

    // Salva os parâmetros para o visualizador Python.
    if (!save_metadata(&sim))
    {
        free_simulation(&sim);
        return EXIT_FAILURE;
    }

    // Exibe as configurações utilizadas.
    print_simulation_info(&sim);

    printf(
        "Modo selecionado: %s\n\n",
        get_mode_name(sim.mode)
    );

    // Executa a simulação.
    if (!run_simulation(&sim))
    {
        free_simulation(&sim);
        return EXIT_FAILURE;
    }

    // Libera a memória utilizada.
    free_simulation(&sim);

    printf(
        "\nSimulacao concluida.\n"
    );

```

```
68      return EXIT_SUCCESS;
69  }
```

D.3 Old/fluid.c

```c
#include "fluid.h"

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <sys/stat.h>
#include <errno.h>
#include <string.h>


// Retorna o índice linear correspondente à posição (i,j).
static inline int index_of(const Simulation *sim, int i, int j)
{
    return i * sim->ny + j;
}


// Aplica condições de contorno periódicas.
static inline int periodic(int value, int size)
{
    if (value < 0)
        return value + size;

    if (value >= size)
        return value - size;

    return value;
}


// Libera toda a memória usada pela simulação.
void free_simulation(Simulation *sim)
{
    free(sim->u);
    free(sim->v);
    free(sim->u_new);
    free(sim->v_new);

    sim->u = NULL;
    sim->v = NULL;
    sim->u_new = NULL;
    sim->v_new = NULL;
}


// Aloca os campos de velocidade.
static int allocate_fields(Simulation *sim)
{
    size_t total =
        (size_t) sim->nx * sim->ny;

    sim->u = calloc(
        total,
        sizeof(double)
    );

    sim->v = calloc(
        total,
        sizeof(double)
    );

    sim->u_new = calloc(
        total,
        sizeof(double)
    );

    sim->v_new = calloc(
```

```
        total,
        sizeof(double)
    );

    if (!sim->u || !sim->v ||
        !sim->u_new || !sim->v_new)
    {
        fprintf(
            stderr,
            "Falha na alocacao de memoria.\n"
        );

        free_simulation(sim);

        return 0;
    }

    return 1;
}


// Inicializa os parâmetros da simulação.
int init_simulation(Simulation *sim, int mode)
{
    sim->nx = NX;
    sim->ny = NY;

    sim->lx = LX;
    sim->ly = LY;

    sim->nu = NU;
    sim->alpha = ALPHA;

    sim->steps = STEPS;
    sim->save_every = SAVE_EVERY;
    sim->mode = mode;

    sim->dx = sim->lx / sim->nx;
    sim->dy = sim->ly / sim->ny;

    sim->dt =
        sim->alpha *
        sim->dx *
        sim->dx /
        sim->nu;

    sim->u = NULL;
    sim->v = NULL;
    sim->u_new = NULL;
    sim->v_new = NULL;

    return allocate_fields(sim);
}


// Define as condições iniciais.
void initialize_velocity(Simulation *sim)
{
    const double cx = 0.5;
    const double cy = 0.5;
    const double sigma = 0.08;

    for (int i = 0; i < sim->nx; ++i)
    {
        double x =
            (i + 0.5) * sim->dx;

        for (int j = 0; j < sim->ny; ++j)
        {
            double y =
```

```
                (j + 0.5) * sim->dy;

            int current =
                index_of(sim, i, j);

            // Inicialmente o fluido está parado.
            sim->u[current] = 0.0;
            sim->v[current] = 0.0;

            // Os modos 1 e 2 possuem velocidade uniforme.
            if (sim->mode >= 1)
            {
                sim->u[current] = 1.0;
            }

            // O modo 2 adiciona uma perturbação gaussiana.
            if (sim->mode == 2)
            {
                double dx = x - cx;
                double dy = y - cy;

                double r2 =
                    dx * dx +
                    dy * dy;

                double perturbation =
                    0.5 *
                    exp(
                        -r2 /
                        (2.0 * sigma * sigma)
                    );

                sim->u[current] +=
                    perturbation;
            }
        }
    }
}


// Cria o diretório dos frames.
int create_frames_directory(void)
{
    if (mkdir("frames", 0755) != 0 &&
        errno != EEXIST)
    {
        fprintf(
            stderr,
            "Erro ao criar diretorio frames: %s\n",
            strerror(errno)
        );

        return 0;
    }

    return 1;
}


// Salva os parâmetros usados pelo Python.
int save_metadata(const Simulation *sim)
{
    FILE *file =
        fopen("metadata.txt", "w");

    if (!file)
    {
        fprintf(
            stderr,
            "Erro ao criar metadata.txt: %s\n",
```

```
            strerror(errno)
        );

        return 0;
    }

    fprintf(
        file,
        "NX %d\n",
        sim->nx
    );

    fprintf(
        file,
        "NY %d\n",
        sim->ny
    );

    fprintf(
        file,
        "LX %.12e\n",
        sim->lx
    );

    fprintf(
        file,
        "LY %.12e\n",
        sim->ly
    );

    fprintf(
        file,
        "NU %.12e\n",
        sim->nu
    );

    fprintf(
        file,
        "DT %.12e\n",
        sim->dt
    );

    fprintf(
        file,
        "STEPS %d\n",
        sim->steps
    );

    fprintf(
        file,
        "SAVE_EVERY %d\n",
        sim->save_every
    );

    fprintf(
        file,
        "MODE %d\n",
        sim->mode
    );

    fprintf(
        file,
        "TFINAL %.12e\n",
        sim->steps * sim->dt
    );

    fclose(file);

    return 1;
}
```

```
278
279
280 // Salva o campo atual em um arquivo CSV.
281 int save_frame(
282     const Simulation *sim,
283     int step
284 )
285 {
286     char filename[256];
287
288     snprintf(
289         filename,
290         sizeof(filename),
291         "frames/frame_%05d.csv",
292         step / sim->save_every
293     );
294
295     FILE *file =
296         fopen(filename, "w");
297
298     if (!file)
299     {
300         fprintf(
301             stderr,
302             "Erro ao abrir %s: %s\n",
303             filename,
304             strerror(errno)
305         );
306
307         return 0;
308     }
309
310     fprintf(
311         file,
312         "i,j,u,v\n"
313     );
314
315     for (int i = 0; i < sim->nx; ++i)
316     {
317         for (int j = 0; j < sim->ny; ++j)
318         {
319             int current =
320                 index_of(sim, i, j);
321
322             fprintf(
323                 file,
324                 "%d,%d,%.12e,%.12e\n",
325                 i,
326                 j,
327                 sim->u[current],
328                 sim->v[current]
329             );
330         }
331     }
332
333     fclose(file);
334
335     return 1;
336 }
337
338
339 // Calcula e imprime as medidas da perturbação.
340 void print_diagnostics(
341     const Simulation *sim,
342     int step
343 )
344 {
345     double max_perturbation = 0.0;
346     double sum_squared = 0.0;
347
```

```
348     double reference_u =
349         sim->mode >= 1 ? 1.0 : 0.0;
350
351     for (int i = 0; i < sim->nx; ++i)
352     {
353         for (int j = 0; j < sim->ny; ++j)
354         {
355             int current =
356                 index_of(sim, i, j);
357
358             double du =
359                 sim->u[current] -
360                 reference_u;
361
362             double dv =
363                 sim->v[current];
364
365             double amplitude =
366                 sqrt(
367                     du * du +
368                     dv * dv
369                 );
370
371             if (amplitude >
372                 max_perturbation)
373             {
374                 max_perturbation =
375                     amplitude;
376             }
377
378             sum_squared +=
379                 du * du +
380                 dv * dv;
381         }
382     }
383
384     double rms =
385         sqrt(
386             sum_squared /
387             ((double) sim->nx *
388              sim->ny)
389         );
390
391     double time =
392         step * sim->dt;
393
394     printf(
395         "step=%4d | "
396         "t=%8.4f | "
397         "max_pert=%10.6e | "
398         "RMS_pert=%10.6e\n",
399         step,
400         time,
401         max_perturbation,
402         rms
403     );
404 }
405
406
407 // Calcula o próximo estado por diferenças finitas. (Ponto Chave)
408 void compute_next_state(Simulation *sim)
409 {
410     double dx2 = sim->dx * sim->dx;
411     double dy2 = sim->dy * sim->dy;
412
413     for (int i = 0; i < sim->nx; ++i) // Loop sobre a dimensão x
414     {
415         int previous_i = periodic(i - 1, sim->nx);
416         int next_i = periodic(i + 1, sim->nx);
417
```

```
418         for (int j = 0; j < sim->ny; ++j)   // Loop sobre a dimensão y
419         {
420             int previous_j = periodic(j - 1, sim->ny);
421             int next_j = periodic(j + 1, sim->ny);
422 
423             int current = index_of(sim, i, j);
424             int left = index_of(sim, previous_i, j);
425             int right = index_of(sim, next_i, j);
426             int down = index_of(sim, i, previous_j);
427             int up = index_of(sim, i, next_j);
428 
429             // Calcula o Laplaciano da componente u.
430             double laplacian_u = (
431                 (sim->u[right] - 2.0 * sim->u[current] + sim->u[left]) / dx2
432                 + (sim->u[up] - 2.0 * sim->u[current] + sim->u[down]) / dy2
433             );
434 
435             // Calcula o Laplaciano da componente v.
436             double laplacian_v = (
437                 (sim->v[right] - 2.0 * sim->v[current] + sim->v[left]) / dx2
438                 + (sim->v[up] - 2.0 * sim->v[current] + sim->v[down]) / dy2
439             );
440 
441             // Aplica Euler explícito à componente u.
442             sim->u_new[current] = sim->u[current] + sim->nu * sim->dt * laplacian_u;
443 
444             // Aplica Euler explícito à componente v.
445             sim->v_new[current] = sim->v[current] + sim->nu * sim->dt * laplacian_v;
446         }
447     }
448 }
449 
450 
451 // Troca os campos atual e novo.
452 void swap_fields(Simulation *sim)
453 {
454     double *temp;
455 
456     temp = sim->u;
457     sim->u = sim->u_new;
458     sim->u_new = temp;
459 
460     temp = sim->v;
461     sim->v = sim->v_new;
462     sim->v_new = temp;
463 }
464 
465 
466 // Executa a evolução temporal.
467 int run_simulation(Simulation *sim)
468 {
469     for (int step = 0;
470          step <= sim->steps;
471          ++step)
472     {
473         // Salva um frame nos intervalos definidos.
474         if (step % sim->save_every == 0)
475         {
476             if (!save_frame(sim, step))
477                 return 0;
478 
479             print_diagnostics(
480                 sim,
481                 step
482             );
483         }
484 
485         // Evita calcular um passo após o estado final.
486         if (step == sim->steps)
487             break;
```

```
488
489         // Calcula o próximo estado.
490         compute_next_state(sim);
491
492         // Torna o novo estado o estado atual.
493         swap_fields(sim);
494     }
495
496     return 1;
497 }
498
499
500 // Imprime as configurações da simulação.
501 void print_simulation_info(
502     const Simulation *sim
503 )
504 {
505     printf(
506         "Modo: %d\n",
507         sim->mode
508     );
509
510     printf(
511         "NX=%d NY=%d | "
512         "nu=%.5f | "
513         "dt=%.8f | "
514         "alpha=%.3f\n",
515         sim->nx,
516         sim->ny,
517         sim->nu,
518         sim->dt,
519         sim->alpha
520     );
521
522     printf(
523         "Tempo final: %.5f\n",
524         sim->steps * sim->dt
525     );
526
527     printf(
528         "Arquivos salvos em ./frames/\n\n"
529     );
530 }
531
532
533 // Retorna uma descrição do modo utilizado.
534 const char *get_mode_name(int mode)
535 {
536     switch (mode)
537     {
538         case 0:
539             return "fluido parado";
540
541         case 1:
542             return "velocidade uniforme";
543
544         case 2:
545             return "velocidade uniforme + perturbacao";
546
547         default:
548             return "modo desconhecido";
549     }
550 }
```

D.4 Old/visualize_fluid.py

```python
import glob
import os
import sys

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation


FRAME_DIR = "frames"
METADATA_FILE = "metadata.txt"


def read_metadata(filename):
    """Lê os parâmetros da simulação armazenados em metadata.txt."""
    meta = {}

    with open(filename, "r", encoding="utf-8") as f:
        for line in f:
            key, value = line.split()
            meta[key] = value

    return meta


def load_frame(filename, nx, ny):
    """Carrega um frame CSV e reconstrói os campos de velocidade u e v."""
    data = np.loadtxt(
        filename,
        delimiter=",",
        skiprows=1
    )

    u = data[:, 2].reshape(nx, ny)
    v = data[:, 3].reshape(nx, ny)

    return u, v


def progress_callback(current_frame, total_frames):
    """
    Exibe o progresso da geração do GIF no terminal.

    current_frame começa em 0, por isso é necessário somar 1
    para mostrar a numeração do frame de forma intuitiva.
    """
    frame = current_frame + 1
    percent = 100.0 * frame / total_frames

    print(
        f"\rGerando GIF: frame {frame}/{total_frames} "
        f"({percent:5.1f}%)",
        end="",
        flush=True
    )

    if frame == total_frames:
        print()


def main():
    # Permite informar uma pasta diferente para os frames pela linha de comando.
    frame_dir = (
        sys.argv[1]
        if len(sys.argv) > 1
        else FRAME_DIR
    )
```

```
68
69      # O arquivo metadata.txt fica no mesmo diretório da pasta de frames.
70      metadata_path = os.path.join(
71          os.path.dirname(frame_dir) or ".",
72          METADATA_FILE
73      )
74
75      if not os.path.exists(metadata_path):
76          raise FileNotFoundError(
77              f"Arquivo '{metadata_path}' não encontrado. "
78              "Execute primeiro o programa em C."
79          )
80
81      # Lê os parâmetros da simulação.
82      meta = read_metadata(metadata_path)
83
84      nx = int(meta["NX"])
85      ny = int(meta["NY"])
86      lx = float(meta["LX"])
87      ly = float(meta["LY"])
88      dt = float(meta["DT"])
89      save_every = int(meta["SAVE_EVERY"])
90      nu = float(meta["NU"])
91      mode = int(meta["MODE"])
92
93      # Localiza todos os arquivos CSV gerados pelo programa em C.
94      files = sorted(
95          glob.glob(
96              os.path.join(
97                  frame_dir,
98                  "frame_*.csv"
99              )
100         )
101     )
102
103     if not files:
104         raise FileNotFoundError(
105             f"Nenhum frame encontrado em '{frame_dir}'. "
106             "Execute primeiro o programa em C."
107         )
108
109     print(f"{len(files)} frames encontrados.")
110
111     # Carrega os frames para a animação e para a análise da perturbação.
112     frames = [
113         load_frame(file, nx, ny)
114         for file in files
115     ]
116
117     # Calcula a magnitude da velocidade em cada frame.
118     speeds = [
119         np.hypot(u, v)
120         for u, v in frames
121     ]
122
123     # Calcula as coordenadas dos centros das células da malha.
124     x = (
125         np.arange(nx) + 0.5
126     ) * lx / nx
127
128     y = (
129         np.arange(ny) + 0.5
130     ) * ly / ny
131
132     X, Y = np.meshgrid(
133         x,
134         y,
135         indexing="ij"
136     )
137
```

```
    # Determina a maior magnitude de velocidade para manter
    # a mesma escala de cores durante toda a animação.
    vmax = max(
        float(np.max(speed))
        for speed in speeds
    )

    vmax = max(vmax, 1e-12)

    # Cria uma figura com o campo de velocidade e o gráfico da perturbação.
    fig, (ax_field, ax_decay) = plt.subplots(
        1,
        2,
        figsize=(13, 5)
    )

    # Mostra a magnitude da velocidade como imagem de fundo.
    image = ax_field.imshow(
        speeds[0].T,
        origin="lower",
        extent=[0, lx, 0, ly],
        aspect="equal",
        vmin=0,
        vmax=vmax,
        interpolation="nearest"
    )

    fig.colorbar(
        image,
        ax=ax_field,
        label="|v| (magnitude da velocidade)"
    )

    # Reduz a quantidade de setas para manter o campo vetorial legível.
    skip = max(1, nx // 20)

    quiver = ax_field.quiver(
        X[::skip, ::skip],
        Y[::skip, ::skip],
        frames[0][0][::skip, ::skip].T,
        frames[0][1][::skip, ::skip].T,
        angles="xy",
        scale_units="xy",
        scale=max(vmax / 0.15, 1.0)
    )

    ax_field.set_xlabel("x")
    ax_field.set_ylabel("y")
    ax_field.set_title("Campo de velocidade")

    # Define o campo uniforme de referência utilizado para medir
    # a intensidade da perturbação.
    background_u = 1.0 if mode >= 1 else 0.0

    # Calcula o instante correspondente a cada frame.
    times = (
        np.arange(len(frames))
        * save_every
        * dt
    )

    rms = []
    max_pert = []

    # Calcula o RMS e o valor máximo da perturbação em cada frame.
    for u, v in frames:
        du = u - background_u
        dv = v

        rms.append(
```

```
            float(
                np.sqrt(
                    np.mean(
                        du**2 + dv**2
                    )
                )
            )
        )

        max_pert.append(
            float(
                np.max(
                    np.hypot(
                        du,
                        dv
                    )
                )
            )
        )

    # Inicializa o gráfico do decaimento da perturbação.
    ax_decay.plot(
        times,
        rms,
        label="RMS da perturbação"
    )

    ax_decay.plot(
        times,
        max_pert,
        label="Máximo da perturbação"
    )

    # Marcadores que indicam a posição atual durante a animação.
    marker_rms, = ax_decay.plot(
        [times[0]],
        [rms[0]],
        marker="o",
        linestyle="None"
    )

    marker_max, = ax_decay.plot(
        [times[0]],
        [max_pert[0]],
        marker="o",
        linestyle="None"
    )

    ax_decay.set_xlabel("Tempo")
    ax_decay.set_ylabel("Amplitude da perturbação")
    ax_decay.set_title("Difusão da perturbação")
    ax_decay.grid(True, alpha=0.3)
    ax_decay.legend()

    # Define o texto correspondente ao modo da simulação.
    if mode == 0:
        mode_text = "fluido parado"
    elif mode == 1:
        mode_text = "velocidade uniforme"
    else:
        mode_text = "velocidade uniforme + perturbação"

    fig.suptitle(
        f"Difusão viscosa | nu={nu:g} | {mode_text}"
    )

    plt.tight_layout()

    def update(frame_number):
        """Atualiza o campo de velocidade e o gráfico da perturbação."""
```

```
        u, v = frames[frame_number]
        speed = speeds[frame_number]

        # Atualiza a imagem da magnitude da velocidade.
        image.set_data(speed.T)

        # Atualiza as setas do campo vetorial.
        quiver.set_UVC(
            u[::skip, ::skip].T,
            v[::skip, ::skip].T
        )

        ax_field.set_title(
            f"Campo de velocidade | "
            f"frame {frame_number + 1}/{len(frames)}"
        )

        # Atualiza os marcadores do gráfico da perturbação.
        marker_rms.set_data(
            [times[frame_number]],
            [rms[frame_number]]
        )

        marker_max.set_data(
            [times[frame_number]],
            [max_pert[frame_number]]
        )

        return (
            image,
            quiver,
            marker_rms,
            marker_max
        )

    # Cria a animação usando todos os frames disponíveis.
    animation = FuncAnimation(
        fig,
        update,
        frames=len(frames),
        interval=60,
        blit=False,
        repeat=True
    )

    print(
        f"\nIniciando geração do GIF com "
        f"{len(frames)} frames..."
    )

    # Salva a animação e exibe o progresso no terminal.
    animation.save(
        "fluid_simulation.gif",
        writer="pillow",
        fps=15,
        progress_callback=progress_callback
    )

    print(
        "GIF salvo em: fluid_simulation.gif"
    )

    plt.close(fig)


if __name__ == "__main__":
    main()
```

E Dados brutos de desempenho e metadados

As 54 medições abaixo correspondem à carga maior descrita na metodologia. Elas são independentes dos 101 frames da execução visual. As médias, os desvios e os speedups já estão apresentados no corpo do relatório.

Tabela 5: Todas as 54 medições, sem exclusão de amostras.

Versão	Schedule	Threads	Repetição	Tempo (s)
normal	static	1	1	5,733336
normal	static	1	2	5,783658
normal	static	1	3	5,749030
normal	dynamic,8	1	1	5,728573
normal	dynamic,8	1	2	5,571651
normal	dynamic,8	1	3	5,485154
normal	guided,8	1	1	5,533111
normal	guided,8	1	2	5,347620
normal	guided,8	1	3	5,473343
normal	static	2	1	2,919736
normal	static	2	2	2,865593
normal	static	2	3	2,911765
normal	dynamic,8	2	1	2,892759
normal	dynamic,8	2	2	2,899603
normal	dynamic,8	2	3	2,929865
normal	guided,8	2	1	2,795917
normal	guided,8	2	2	2,823405
normal	guided,8	2	3	2,783449
normal	static	4	1	2,538934
normal	static	4	2	1,815439
normal	static	4	3	1,808791
normal	dynamic,8	4	1	1,738677
normal	dynamic,8	4	2	1,733609
normal	dynamic,8	4	3	1,858730
normal	guided,8	4	1	1,724903
normal	guided,8	4	2	1,760039
normal	guided,8	4	3	1,800524
collapse	static	1	1	6,107416
collapse	static	1	2	6,012373
collapse	static	1	3	6,004658
collapse	dynamic,8	1	1	9,762984
collapse	dynamic,8	1	2	9,772458
collapse	dynamic,8	1	3	9,742319
collapse	guided,8	1	1	6,083705
collapse	guided,8	1	2	6,131047
collapse	guided,8	1	3	6,083059
collapse	static	2	1	3,170191
collapse	static	2	2	3,213320

Versão	Schedule	Threads	Repetição	Tempo (s)
collapse	static	2	3	3,249207
collapse	dynamic,8	2	1	9,377309
collapse	dynamic,8	2	2	9,525541
collapse	dynamic,8	2	3	9,611482
collapse	guided,8	2	1	3,156402
collapse	guided,8	2	2	3,114199
collapse	guided,8	2	3	3,195055
collapse	static	4	1	2,009164
collapse	static	4	2	2,625166
collapse	static	4	3	2,040834
collapse	dynamic,8	4	1	6,734208
collapse	dynamic,8	4	2	7,150051
collapse	dynamic,8	4	3	6,787196
collapse	guided,8	4	1	1,957294
collapse	guided,8	4	2	1,914954
collapse	guided,8	4	3	1,956753

E.1 Configuração dos testes

```
MODO 2
NX 400
NY 400
STEPS 5000
REPETICOES 3
```

E.2 Metadados da execução visual

```
NX 80
NY 80
LX 1.000000000000e+00
LY 1.000000000000e+00
NU 1.000000000000e-02
DT 3.125000000000e-03
STEPS 1000
SAVE_EVERY 10
MODE 2
TFINAL 3.125000000000e+00
```

F Série completa dos diagnósticos

A tabela contém uma linha por frame salvo. A variância é calculada em relação à média do próprio campo. A componente v permaneceu nula em todos os frames; o erro máximo de conservação da média foi inferior a 4×10^{-14}.

Tabela 6: Série completa dos 101 estados salvos no modo 2. A média de u se conserva em aproximadamente 1,020106192967 e a componente v é nula em todos os estados.

Passo	t	Máximo	RMS	Variância de u
0	0,00000	0,49695754	0,07089815	0,0046222893
10	0,03125	0,45285609	0,06766583	0,0041744050
20	0,06250	0,41596257	0,06483950	0,0037999017
30	0,09375	0,38463974	0,06234061	0,0034820923
40	0,12500	0,35771222	0,06011037	0,0032089975
50	0,15625	0,33431409	0,05810381	0,0029717933
60	0,18750	0,31379317	0,05628583	0,0027638359
70	0,21875	0,29564885	0,05462862	0,0025800270
80	0,25000	0,27949041	0,05310977	0,0024163889
90	0,28125	0,26500843	0,05171104	0,0022697724
100	0,31250	0,25195463	0,05041737	0,0021376517
110	0,34375	0,24012750	0,04921622	0,0020179775
120	0,37500	0,22936178	0,04809707	0,0019090695
130	0,40625	0,21952061	0,04705099	0,0018095369
140	0,43750	0,21048973	0,04607036	0,0017182187
150	0,46875	0,20217296	0,04514861	0,0016341382
160	0,50000	0,19448878	0,04428009	0,0015564676
170	0,53125	0,18736762	0,04345987	0,0014845009
180	0,56250	0,18074976	0,04268362	0,0014176322
190	0,59375	0,17458364	0,04194756	0,0013553386
200	0,62500	0,16882452	0,04124834	0,0012971666
210	0,65625	0,16343336	0,04058300	0,0012427207
220	0,68750	0,15837599	0,03994889	0,0011916548
230	0,71875	0,15362233	0,03934366	0,0011436646
240	0,75000	0,14914581	0,03876519	0,0010984813
250	0,78125	0,14492286	0,03821159	0,0010558669
260	0,81250	0,14093254	0,03768115	0,0010156099
270	0,84375	0,13715613	0,03717231	0,0009775216
280	0,87500	0,13357687	0,03668368	0,0009414330
290	0,90625	0,13017972	0,03621397	0,0009071928
300	0,93750	0,12695111	0,03576204	0,0008746646
310	0,96875	0,12387881	0,03532682	0,0008437254
320	1,00000	0,12095174	0,03490735	0,0008142642
330	1,03125	0,11815982	0,03450274	0,0007861801
340	1,06250	0,11549391	0,03411218	0,0007593815
350	1,09375	0,11294567	0,03373491	0,0007337850

Passo	t	Máximo	RMS	Variância de u
360	1,12500	0,11050746	0,03337025	0,0007093146
370	1,15625	0,10817233	0,03301757	0,0006859006
380	1,18750	0,10593385	0,03267627	0,0006634793
390	1,21875	0,10378616	0,03234580	0,0006419921
400	1,25000	0,10172384	0,03202568	0,0006213851
410	1,28125	0,09974190	0,03171542	0,0006016087
420	1,31250	0,09783573	0,03141459	0,0005826172
430	1,34375	0,09600107	0,03112277	0,0005643681
440	1,37500	0,09423397	0,03083960	0,0005468221
450	1,40625	0,09253076	0,03056472	0,0005299430
460	1,43750	0,09088805	0,03029778	0,0005136967
470	1,46875	0,08930267	0,03003849	0,0004980519
480	1,50000	0,08777168	0,02978654	0,0004829792
490	1,53125	0,08629232	0,02954167	0,0004684510
500	1,56250	0,08486203	0,02930360	0,0004544419
510	1,59375	0,08347842	0,02907210	0,0004409278
520	1,62500	0,08213923	0,02884693	0,0004278862
530	1,65625	0,08084238	0,02862787	0,0004152960
540	1,68750	0,07958588	0,02841472	0,0004031373
550	1,71875	0,07836789	0,02820728	0,0003913915
560	1,75000	0,07718667	0,02800536	0,0003800410
570	1,78125	0,07604060	0,02780878	0,0003690691
580	1,81250	0,07492812	0,02761737	0,0003584601
590	1,84375	0,07384780	0,02743097	0,0003481993
600	1,87500	0,07279827	0,02724943	0,0003382726
610	1,90625	0,07177825	0,02707260	0,0003286667
620	1,93750	0,07078650	0,02690034	0,0003193691
630	1,96875	0,06982189	0,02673250	0,0003103676
640	2,00000	0,06888333	0,02656897	0,0003016511
650	2,03125	0,06796979	0,02640961	0,0002932087
660	2,06250	0,06708029	0,02625431	0,0002850300
670	2,09375	0,06621391	0,02610296	0,0002771054
680	2,12500	0,06536977	0,02595543	0,0002694254
690	2,15625	0,06454705	0,02581163	0,0002619812
700	2,18750	0,06374495	0,02567145	0,0002547643
710	2,21875	0,06296272	0,02553479	0,0002477666
720	2,25000	0,06219965	0,02540156	0,0002409803
730	2,28125	0,06145506	0,02527166	0,0002343979
740	2,31250	0,06072830	0,02514501	0,0002280125
750	2,34375	0,06001876	0,02502151	0,0002218172
760	2,37500	0,05932584	0,02490109	0,0002158055
770	2,40625	0,05864900	0,02478367	0,0002099712
780	2,43750	0,05798769	0,02466916	0,0002043082
790	2,46875	0,05734141	0,02455748	0,0001988109
800	2,50000	0,05670967	0,02444857	0,0001934738
810	2,53125	0,05609200	0,02434236	0,0001882915

Passo	t	Máximo	RMS	Variância de u
820	2,56250	0,05548795	0,02423877	0,0001832590
830	2,59375	0,05489711	0,02413774	0,0001783714
840	2,62500	0,05431906	0,02403920	0,0001736240
850	2,65625	0,05375342	0,02394308	0,0001690123
860	2,68750	0,05319980	0,02384934	0,0001645320
870	2,71875	0,05265785	0,02375790	0,0001601788
880	2,75000	0,05212723	0,02366871	0,0001559487
890	2,78125	0,05160761	0,02358171	0,0001518380
900	2,81250	0,05109867	0,02349685	0,0001478428
910	2,84375	0,05060010	0,02341407	0,0001439595
920	2,87500	0,05011161	0,02333332	0,0001401847
930	2,90625	0,04963293	0,02325455	0,0001365150
940	2,93750	0,04916378	0,02317771	0,0001329472
950	2,96875	0,04870390	0,02310275	0,0001294783
960	3,00000	0,04825304	0,02302963	0,0001261051
970	3,03125	0,04781096	0,02295830	0,0001228248
980	3,06250	0,04737744	0,02288872	0,0001196346
990	3,09375	0,04695224	0,02282084	0,0001165317
1000	3,12500	0,04653515	0,02275462	0,0001135137

G Código completo da geração de figuras e tabelas

O script a seguir trabalha sobre os dados já coletados; não executa uma nova simulação nem produz novas medições de desempenho.

```
"""Atualiza figuras e tabela a partir dos dados incluídos, sem executar a simulação."""
from pathlib import Path
import csv
from collections import defaultdict
from statistics import mean, stdev

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np


ROOT = Path(__file__).resolve().parent
DATA = ROOT / "dados"
FIG = ROOT / "figuras"
TABLE = ROOT / "tabelas"
FIG.mkdir(exist_ok=True)
TABLE.mkdir(exist_ok=True)
plt.rcParams.update({"font.size": 10, "axes.spines.top": False,
                     "axes.spines.right": False, "pdf.fonttype": 42})


def read_csv(name):
    with (DATA / name).open(encoding="utf-8") as file:
        return list(csv.DictReader(file, delimiter=";"))


def pt(value, digits=3):
    return f"{value:.{digits}f}".replace(".", ",")


groups = defaultdict(list)
for row in read_csv("tempos.csv"):
    groups[row["versao"], int(row["threads"]), row["schedule"]].append(
        float(row["tempo_segundos"]))
assert len(groups) == 18 and all(len(values) == 3 for values in groups.values())
baseline = mean(groups["normal", 1, "static"])
versions = ("normal", "collapse")
schedules = ("static", "dynamic,8", "guided,8")
threads = (1, 2, 4)
colors = ("#2366a8", "#d4791f", "#258563")
stats = {}
for key, values in groups.items():
    avg, sd = mean(values), stdev(values)
    stats[key] = (avg, sd, baseline / avg, 100 * sd / avg)

for name, speedup in (("tempos", False), ("speedup", True)):
    fig, axes = plt.subplots(1, 2, figsize=(8.6, 3.8), sharey=True,
                             layout="constrained")
    for ax, version in zip(axes, versions):
        for schedule, color in zip(schedules, colors):
            averages = [stats[version, p, schedule][0] for p in threads]
            deviations = [stats[version, p, schedule][1] for p in threads]
            if speedup:
                values = [baseline / value for value in averages]
                ax.plot(threads, values, "o-", color=color, label=schedule)
            else:
                ax.errorbar(threads, averages, yerr=deviations, fmt="o-",
                            capsize=4, color=color, label=schedule)
                # Mostra as três medições, inclusive as que mais se afastam.
                for p in threads:
                    ax.scatter([p] * 3, groups[version, p, schedule],
                               color=color, alpha=.38, s=17, zorder=3)
```

```
        if speedup:
            ax.plot(threads, threads, "--", color="0.55", label="Referência ideal")
        ax.set_title("Sem collapse" if version == "normal" else "Com collapse(2)")
        ax.set_xlabel("Threads solicitadas")
        ax.set_xticks(threads)
        ax.grid(alpha=.22)
        ax.legend(fontsize=8)
    upper = 4.4 if speedup else 1.12 * max(max(values) for values in groups.values())
    axes[0].set_ylim(0, upper)
    axes[0].set_ylabel("Speedup sobre normal / 1 / static" if speedup
                       else "Tempo de cálculo (s)")
    fig.savefig(FIG / f"{name}.pdf", bbox_inches="tight")
    plt.close(fig)

validation = read_csv("validacao_numerica.csv")
t = np.array([float(row["tempo_fisico"]) for row in validation])
maximum = [float(row["max_pert"]) for row in validation]
rms = [float(row["rms_pert"]) for row in validation]
limit = float(validation[0]["media_u"]) - 1
fig, ax = plt.subplots(figsize=(8.4, 3.6), layout="constrained")
ax.plot(t, maximum, label="Máximo da perturbação", color=colors[1])
ax.plot(t, rms, label="RMS da perturbação", color=colors[0])
ax.axhline(limit, ls="--", color="0.45", label="Média da perturbação conservada")
ax.set(xlabel="Tempo físico simulado", ylabel="Amplitude", ylim=(0, .52))
ax.grid(alpha=.22)
ax.legend()
fig.savefig(FIG / "decaimento.pdf", bbox_inches="tight")
plt.close(fig)

fig, axes = plt.subplots(1, 2, figsize=(8.6, 3.7), layout="constrained")
for ax, filename, title in zip(axes, ("campo_inicial.csv", "campo_final.csv"),
                               ("Estado inicial (t = 0)", "Estado final (t = 3,125)")):
    array = np.loadtxt(DATA / filename, delimiter=",", skiprows=1)
    u = array[:, 2].reshape(80, 80)
    image = ax.imshow((u - 1).T, origin="lower", extent=(0, 1, 0, 1),
                      vmin=0, vmax=.5, cmap="viridis")
    ax.set(title=title, xlabel="x", ylabel="y")
fig.colorbar(image, ax=axes, label="Perturbação u − 1", shrink=.85)
fig.savefig(FIG / "campos.pdf", bbox_inches="tight")
plt.close(fig)

lines = [r"\begin{longtable}{llrrrrr}",
         r"\caption{Resultados da carga de $400\times400$ células e 5.000 passos."
         r" $s$ é o desvio padrão amostral; CV é o coeficiente de variação.}"
         r"\label{tab:resultados}\\",
         r"\toprule Versão & Schedule & Threads & Média (s) & $s$ (s) & Speedup & CV (\%) \\",
         r"\midrule\endfirsthead",
         r"\toprule Versão & Schedule & Threads & Média (s) & $s$ (s) & Speedup & CV (\%) \\",
         r"\midrule\endhead", r"\bottomrule\endfoot"]
for version in versions:
    for p in threads:
        for schedule in schedules:
            avg, sd, speed, cv = stats[version, p, schedule]
            label = "Normal" if version == "normal" else "Collapse"
            lines.append(f"{label} & \\texttt{{{schedule}}} & {p} & "
                         f"{pt(avg)} & {pt(sd)} & {pt(speed)} & {pt(cv, 2)} " + r"\\")
    if version == "normal":
        lines.append(r"\midrule")
lines.append(r"\end{longtable}")
(TABLE / "resultados.tex").write_text("\n".join(lines) + "\n", encoding="utf-8")
with (DATA / "estatisticas.csv").open("w", encoding="utf-8", newline="") as file:
    writer = csv.writer(file, delimiter=";", lineterminator="\n")
    writer.writerow(("versao", "threads", "schedule", "media", "desvio_amostral",
                     "speedup_referencia_comum", "cv_percentual"))
    for key, values in stats.items():
        writer.writerow((*key, *values))
print("Figuras, tabela e estatísticas atualizadas a partir dos CSVs incluídos.")

# Registros completos para que os apêndices possam ser lidos somente no PDF.
def longtable(filename, columns, heading, caption, rows):
```

```
    content = [r"\begin{longtable}{" + columns + "}",
               r"\caption{" + caption + r"}\\", r"\toprule " + heading + r"\\",
               r"\midrule\endfirsthead", r"\toprule " + heading + r"\\",
               r"\midrule\endhead", r"\bottomrule\endfoot"]
    content.extend(" & ".join(row) + r"\\" for row in rows)
    content.append(r"\end{longtable}")
    (TABLE / filename).write_text("\n".join(content) + "\n", encoding="utf-8")

longtable("tempos_brutos.tex", "llrrr",
          r"Versão & Schedule & Threads & Repetição & Tempo (s)",
          "Todas as 54 medições, sem exclusão de amostras.",
          [[row["versao"], row["schedule"], row["threads"], row["repeticao"],
            pt(float(row["tempo_segundos"]), 6)] for row in read_csv("tempos.csv")])
longtable("validacao_completa.tex", "rrrrr",
          r"Passo & $t$ & Máximo & RMS & Variância de $u$",
          "Série completa dos 101 estados salvos no modo 2. "
          "A média de $u$ se conserva em aproximadamente $1{,}020106192967$ "
          "e a componente $v$ é nula em todos os estados.",
          [[row["passo"], pt(float(row["tempo_fisico"]), 5),
            pt(float(row["max_pert"]), 8), pt(float(row["rms_pert"]), 8),
            pt(float(row["variancia_u"]), 10)] for row in validation])
```

Referências

[1] OPENMP ARCHITECTURE REVIEW BOARD. *OpenMP Application Programming Interface, Version 5.2*. 2021. Disponível em: `https://www.openmp.org/specifications/`.

[2] PINTO, Matheus Bezeril. *ProgParalela2026.2: Tarefa 11*. Código e resultados da revisão `5192499`, 2026. Disponível em: `https://github.com/39Matheus/ProgParalela2026.2/tree/5192499/Tarefa_11`.